



CENG 408

Innovative System Design and Development II

2025-2026 Spring

**MULTI-LABEL CHEST X-RAY DISEASE
CLASSIFICATION AND EXPLAINABILITY WITH
DEEP LEARNING**

PROJECT REPORT 408

CEVDET ALP ÖZGÜN 202211406

SALİH BARKIN AKKAYA 202211004

İPEK BAŞ 202211013

YAREN AKDOĞAN 202111046

SEZİN KÖROĞLU 202211046

İBRAHİM TAŞPINAR 202111072

Abstract

In today's medical field, chest X-ray images have an indispensable role in the diagnosis of thoracic diseases such as pneumonia, cardiomegaly and atelectasis. Quick and accurate diagnosis is crucial but the growing number of patients and the limited availability of radiologists can delay the results of X-rays. Deep learning techniques such as convolutional neural networks have shown great potential in the automation of disease detection thanks to their ability to detect multiple diseases and then allowing for models with accuracies comparable to human professionals. However, high accuracy is not the only relevant factor for real world clinical adoption. Doctors need to understand why a model makes a certain decision in order to trust and effectively use it. This is where explainable artificial intelligence methods such as Grad-CAM play an important role by visually highlighting the parts of an image that are relevant to the model's prediction through heatmaps. In this report, we explain the experiments we conducted during the development of our model, the challenges we faced during development, the methods we employed in the final version and the results we obtained.

Table of Contents

Abstract	2
1. Introduction.....	6
2. Methodology	7
2.1. Dataset & Preprocessing Methodology	7
2.1.1. NIH ChestX-ray14	7
2.1.2. Dataset Filtering (PA view only)	7
2.1.3. Preprocessing Pipelines.....	8
2.1.4. Standardization for the Backend	10
2.2. Model Architecture And Training.....	10
2.2.1. DenseNet-121 Backbone.....	10
2.2.2. Convolutional Block Attention Module (CBAM).....	11
2.2.3. Model Explainability Using Grad-CAM.....	12
2.2.4. Model Training and Loss Functions.....	12
2.3. Backend & Database Methodology with MongoDB.....	13
2.3.1. Backend Role in the System.....	13
2.3.2. API Design Decisions	14
2.3.3. Why MongoDB?	14
2.3.4. Data Stored in MongoDB.....	14
2.3.5. File Storage Strategy.....	15
2.3.6. Inference Workflow	15
2.3.7. Explainability Workflow (Grad-CAM)	16
2.4. Backend	16
2.4.1. Frontend Interaction and API Integration	16
2.5. Frontend	18
2.5.1. Frontend Interaction and User Interface Design.....	18
2.5.2. User Flow and Application Structure	18
2.5.3. Result Visualization and Grad-CAM Integration.....	18
2.5.4. API Communication and Data Handling	18
2.5.5. Error Handling and User Feedback.....	18
3. Results and Experiments	19
3.1. Model Structure Overview	19
3.1.1. DenseNet-121 Backbone	19
3.1.2. CBAM (Convolutional Block Attention Module)	19
3.2. Model 1: Baseline BCEWithLogitsLoss:.....	20
3.2.1. Training and Validation Phase:	20

3.2.2.	Testing Phase and Performance Evaluation:	21
3.2.3.	In-Depth Analysis and Findings:	22
3.2.4.	Conclusion for Model 1	22
3.3.	Model 2: Asymmetric Loss (ASL) Implementation:	23
3.3.1.	Training and Validation Phase:	23
3.3.2.	Testing Phase and Performance Evaluation	24
3.3.3.	In-Depth Analysis and Comparative Findings:	25
3.3.4.	Conclusion for Model 2 (B2 Level)	25
3.4.	Model 3: ASL with Image Preprocessing (CLAHE + Gamma Correction):	26
3.4.1.	Training and Validation Phase:	26
3.4.2.	Testing Phase and Performance Evaluation	27
3.4.3.	In-Depth Analysis and Comparative Findings	28
3.4.4.	Conclusion for Model 3:	28
3.5.	Model 4: Asymmetric Loss with Dynamic Class-Specific Thresholds (The Final Model)	29
3.5.1.	Training and Validation Phase	29
3.5.2.	Testing Phase and Performance Evaluation	30
3.5.3.	In-Depth Analysis and Final Comparative Findings	31
3.5.4.	Final Conclusion:	32
3.6.	Comparison of Models:	32
3.7.	Experiments:	33
3.7.1.	Example 1	34
3.7.2.	Example 2	36
3.7.3.	Example 3	38
3.8.	Conclusion of Experimental Results	40
4.	Discussion	41
4.1.	Interpretation and Comparison of Results	41
4.1.1.	The Failure of Standard BCE (Model 1)	41
4.1.2.	The Impact of Asymmetric Loss (Model 2)	41
4.1.3.	Preprocessing Limitations (Model 3)	41
4.1.4.	The Success of Dynamic Thresholds (Model 4)	41
4.2.	Limitations	42
4.3.	Ethical Concerns:	42
4.4.	Scalability	42
5.	Conclusion & Future Work	43
5.1.	Model Architecture and Implementation	43
5.1.1.	DenseNet-121 Backbone	43

5.1.2.	Convolutional Block Attention Module	43
5.1.3.	Lessons Learned	43
5.1.4.	Future Work	44
5.2.	Visualization with GradCAM++	44
5.2.1.	Attribution Mechanics and Hook Implementation.....	44
5.2.2.	API Segmentation and Contour Generation	45
5.2.3.	Lessons Learned	45
5.2.4.	Future Work	46
5.3.	Model Training & Dataset	46
5.3.1.	Lessons Learned	47
5.3.2.	Future Work	47
5.4.	Frontend	48
5.4.1.	Role Selection.....	48
5.4.2.	Doctor Registration and Pending Approval Mechanism	49
5.4.3.	Doctor Registration and Pending Approval Mechanism	50
5.4.4.	End-to-End Clinical Workflow and Patient Management.....	51
5.4.5.	Real -Time Neural Processing and Feedback Mechanisms	52
5.4.6.	Explainable AI (XAI) and Diagnostic Visualization	52
5.4.7.	Seamless Diagnostic Reporting and Patient Record Management	53
5.4.8.	Integrated Digital Archiving and Diagnostic History.....	54
5.4.9.	Lessons Learned	55
5.4.10.	Future Work	55
5.5.	Database Design and Implementation	56
5.5.1.	Applications	57
5.5.2.	Lessons Learned	58
5.5.3.	Future Work	59
6.	Conclusion	60
7.	References	61

1. Introduction

Chest X-ray imaging serves as a fundamental diagnostic tool in modern medicine for identifying various thoracic conditions such as pneumonia, cardiomegaly, and atelectasis. However, the increasing volume of medical images and the limited availability of expert radiologists often lead to delays in diagnosis. To address this challenge, deep learning techniques, particularly Convolutional Neural Networks (CNNs), have been widely adopted for automating multi-label disease detection.

While high accuracy is essential, clinical integration requires that these models are also interpretable and trustworthy for healthcare professionals. In this project we have implemented a system that utilizes a DenseNet-121 backbone integrated with Convolutional Block Attention Modules (CBAM) to improve feature extraction from complex radiographic patterns. To provide explainability, GradCAM++ is used to provide visual explanations of the model's decisions through hollow contours generated using the heatmap outputs of GradCAM++. The final monile application was made using Node.js and uses MongoDB as the backend database.

2. Methodology

2.1. Dataset & Preprocessing Methodology

2.1.1. NIH ChestX-ray14

Selection and the analysis of the dataset for this project, we selected the NIH ChestX-ray14 dataset, which is a standard for thoracic disease classification. We chose this dataset because it provides a sufficient amount of collection (112,120 chest X-ray images) which are labeled for 14 different conditions. During our analysis, we observed that the dataset has a multi-label structure. That means a patient could have multiple diseases at the same time.

We also identified a significant class imbalance issue. While common diseases like Infiltration had over 20,000 examples. Rare diseases like Hernia had very few samples. To prevent the model from ignoring these rare classes, we decided to create a weighted learning strategy for our training process. [1]

2.1.2. Dataset Filtering (PA view only)

Before starting the training process, we decided to filter the dataset based on the position of X-Ray views. The original dataset contains both Anterior-Posterior (AP) and Posterior-Anterior (PA) views. We chose to limit our dataset to PA views only, as they usually offer better standardization for deep learning models. We implemented a script that scanned the metadata and selected only the rows where the position of data was labeled as "PA". Based on this filtered list, we recreated the file directory and updated the training/testing split to ensure that no AP images were included in our final dataset.

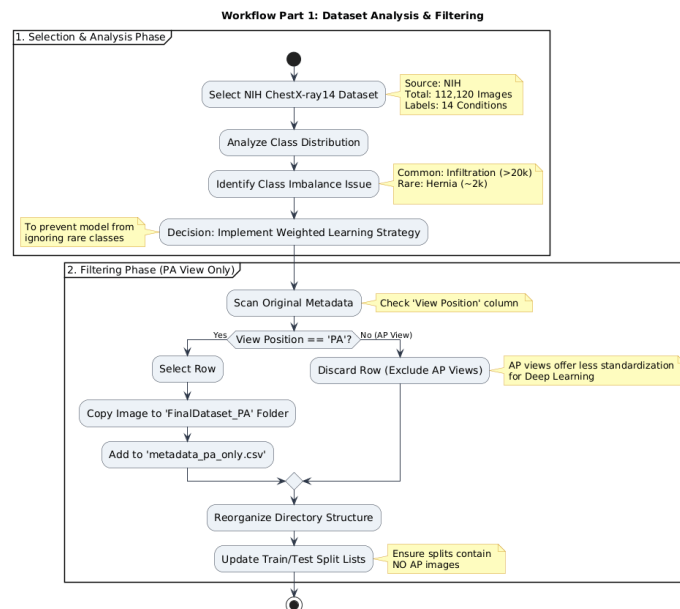


Figure-1: Dataset Analysis and Filtering Workflow Diagram

2.1.3. Preprocessing Pipelines

Since unprocessed X-ray images generally contain noise or varying lighting conditions, we designed four different preprocessing scenarios to improve the model's performance.

2.1.3.1. *Preprocessing Methods That Have Been Implemented*

These are the processing methods we have already implemented to see their effects on the results of our model training and to see whether or not they are suitable for the full application.

- **Intensity Enhancement:** In the first scenario, we focused on improving contrast. We applied Gamma Correction and Contrast Limited Adaptive Histogram Equalization (CLAHE) to make the details of lungs more visible. We also added an unsharp mask to make clear the edges [3],[5]. This method ultimately proved slightly less effective than simply using the images as they were found in the dataset and was discarded.

2.1.3.2. *Preprocessing Methods That Will Be Implemented*

These are the processing methods we are planning to implement after reaching a sufficient stage of development, where we can focus on minor improvements. We have applied these methods on a smaller scale to a limited number of images; however, we have not retrained the model or processed the full dataset using them.

- **Noise Reduction:** In the second scenario, we decided to reduce noise. We used a median filter to make the image smoother while preserving edges. Also, by CLAHE and a sharpening kernel, we highlighted the desired structures [3],[4].
- **Bone Suppression:** We experimented with a bone suppression technique to prevent the ribs from hiding lung abnormalities. We achieved this by subtracting a blurred version of the image from the original. That caused reducing the brightness of the bone structures and made the soft tissue clearer [3].
- **Frequency Domain Filtering:** Finally, we tested a method combining CLAHE with a Butterworth band-pass filter. This allowed us to emphasize specific structural details in the frequency domain while removing high-frequency noise [2],[3].

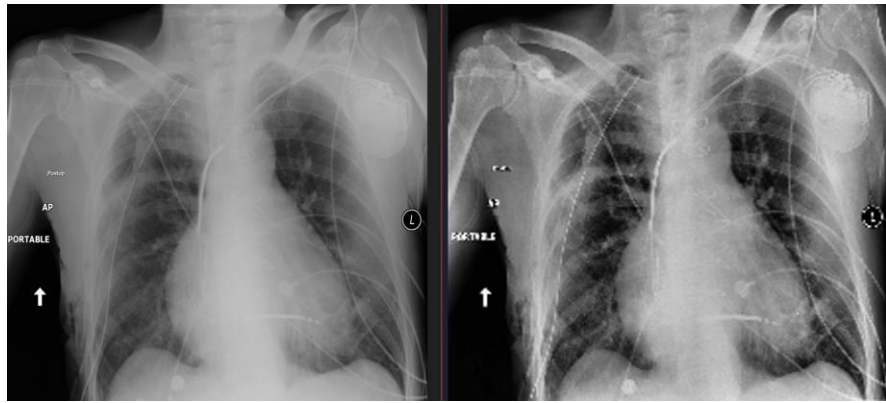


Figure-2: Intensity Enhancement

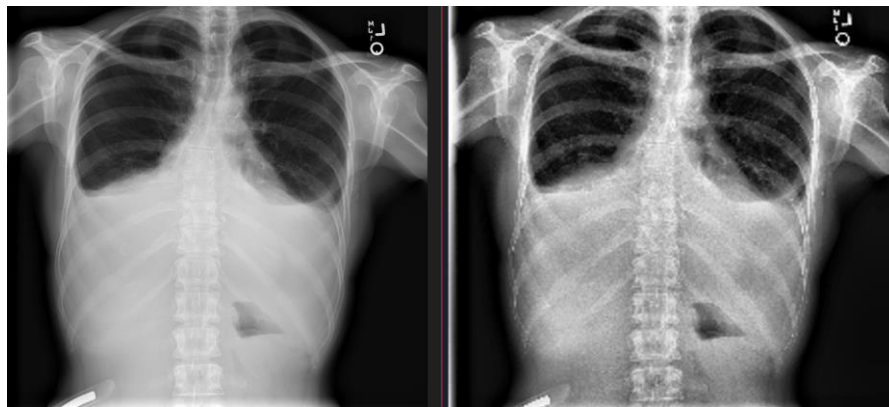


Figure-3: Noise Reduction



Figure-4: Bone Suppression



Figure-5: Frequency Domain Filtering

2.1.4. Standardization for the Backend

Independent of the chosen preprocessing strategy, we decided to put our processed dataset in a standard format. This method ensured the compatibility with our backend system. For every image in our dataset, we resized the resolution to 224x224 pixels. Even though the X-rays images in our dataset are originally in grayscale format we converted them into a 3-channel RGB format by repeating the grayscale channel three times. This decision was made to give us the opportunity to use pre-trained deep learning architectures (DenseNet, ResNet, etc.) that expect 3-channel input by default.

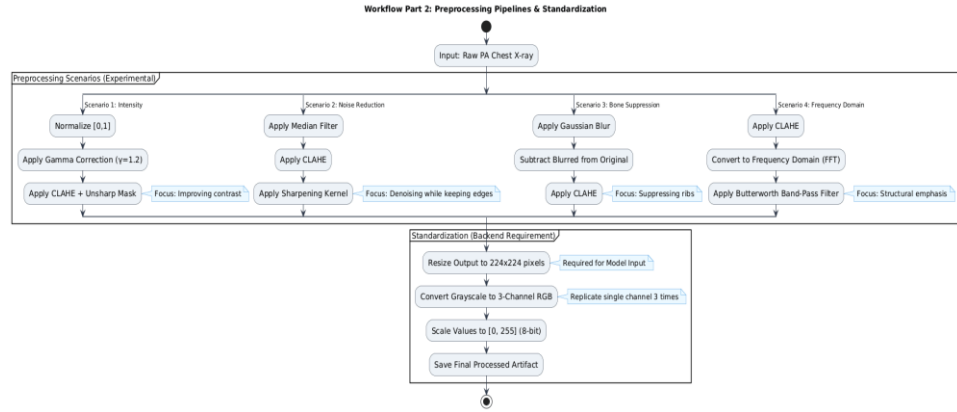


Figure-6: Preprocessing Pipelines and Workflow Diagram

2.2. Model Architecture And Training

2.2.1. DenseNet-121 Backbone

We have chosen DenseNet as the backbone of our model due to its proven success in medical image classification tasks. The dense connectivity technique used in DenseNet allows the outputs of previous layers to be utilized in all future layers within a dense block in the model, which mitigates the possible vanishing gradient issues we may encounter [6]. The pretrained ImageNet weights of DenseNet also allow for faster convergence and stable training. The segmented structure of DenseNet allows for the insertion of different modules between its dense blocks, which is perfect for our project. The dense blocks are made up of bottleneck layers with dense connectivity and make up a large part of the model. Bottleneck layers start with batch normalization and ReLU followed by a 1x1 convolutional layer, after which another batch normalization and ReLU, and end with one final 3x3 convolutional layer [6].

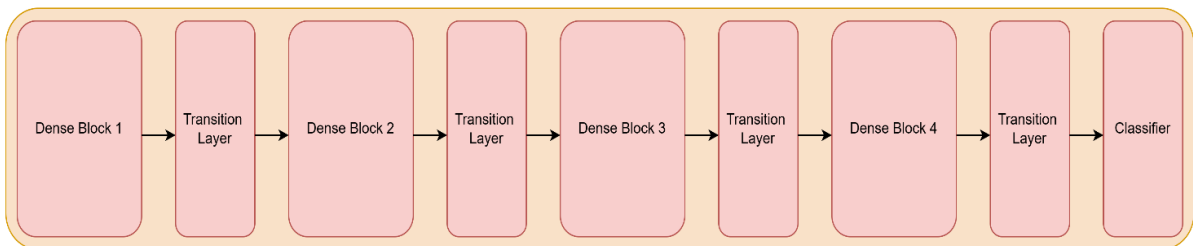


Figure-7: High Level Architecture of DenseNet-121

2.2.2. Convolutional Block Attention Module (CBAM)

CBAM is an attention module comprised of two smaller components called channel attention module and spatial attention module respectively [7]. Neither component affects the input or output sizes of the layers before and after them thus allowing for easy insertion into our model.

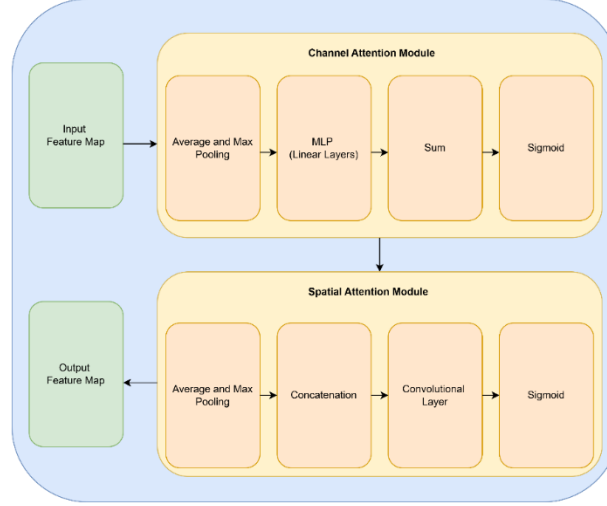


Figure-8: High Level Architecture of CBAM

2.2.2.1. Channel Attention Module

The channel attention module decides what features are important for a given task. The module applies average and max pooling to model the current features and passes them to an MLP. The sigmoid of the sum of the resulting features is multiplied with the current model features to update the feature map of the model [7].

2.2.2.2. Spatial Attention Module

The spatial attention module focuses on where important regions are on the feature maps. Similarly to the channel attention module, it applies average and max pooling across channels to generate maps that highlight where important features are. These feature maps are concatenated to make a feature map with two channels which is passed through a convolutional layer without changing its size while reducing the channel count back down to one. The sigmoid of the resulting map is multiplied with the current feature map to highlight important spatial regions [7].

2.2.2.3. Overall Proposed Architecture

Our model adds a CBAM module after each dense block without changing the feature map size to avoid compatibility issues with subsequent dense blocks. By adding this module after each dense block, we aim to create a model that is more capable of detecting the small differences present in X-ray images without causing an excessive processing load on devices. Due to the precise nature of diagnosing thoracic diseases from X-ray images, we aim to optimize our model's confidence and interpretability. We are planning on using F1 and F1 macro as our metrics during development to evaluate the performance of our model.

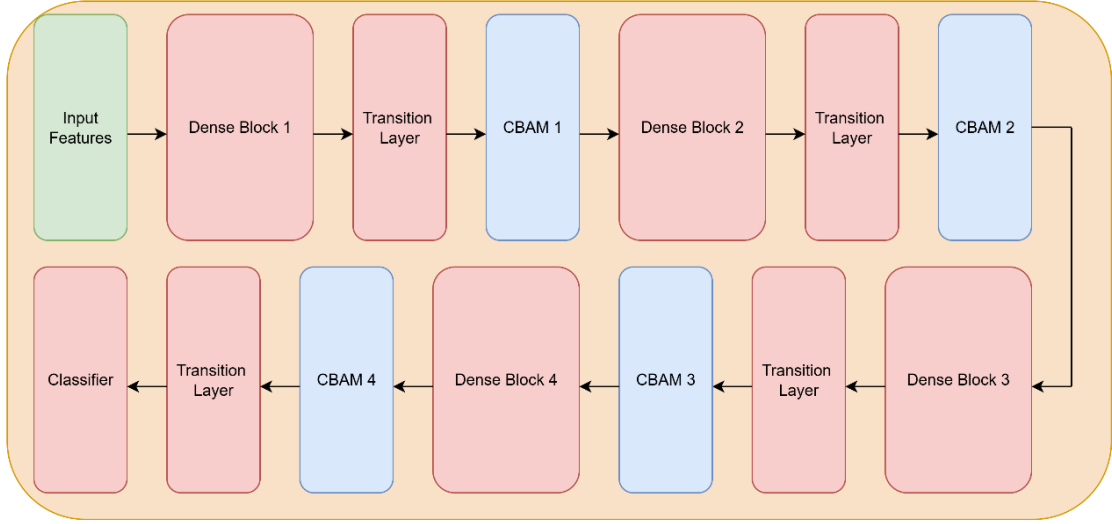


Figure-9: High Level Architecture of Our Model

2.2.3. Model Explainability Using Grad-CAM

Grad-CAM will be integrated into the model outputs to improve model interpretability. Grad-CAM will be used to visualize the areas that are relevant to the predictions our model makes by generating individual heatmaps over the original X-ray image for each possible disease class [8]. Through this technique, we aim to increase the trust of users by giving them the ability to see why the model came to a decision.



Figure-10: Original X-ray Image and The Grad-CAM Heatmap of The Same Image

2.2.4. Model Training and Loss Functions

The model training was designed for the multi-label nature of the dataset and to address the class imbalance across disease categories. Since each chest X-ray image may contain multiple pathologies simultaneously, the training methods and loss functions were selected with multi-label classification in mind.

2.2.4.1. *Binary Cross Entropy with Logits Loss (BCEWithLogitLoss)*

We used Binary Cross Entropy with Logits Loss as our initial baseline for our model. This loss function combines a sigmoid activation with binary cross entropy in a single equation and is widely used for multi-label classification tasks.

However, BCEWithLogitsLoss does not provide a way to get around the sample imbalance issue of our dataset. For the NIH ChestX-ray14 dataset, this can lead to the model performing much worse on rare disease classes.

2.2.4.2. *Asymmetric Loss (ASL)*

To overcome the class imbalance limitations of our baseline, we implemented Asymmetric Loss (ASL) as an alternative. ASL is specifically designed for imbalanced multi-label classification and can internally reduce the dominance of prevalent classes and allow for the model to learn less common ones.

The loss function applies asymmetric focusing to positive and negative samples, allowing the model to assign less importance to negative examples while emphasizing harder and more informative samples.

By emphasizing difficult examples and decreasing the weight of weighting common negatives, ASL improves the model's ability to detect subtle and rare thoracic diseases.

2.2.4.3. *Training Configuration And Hyperparameters*

Transfer learning was used by initializing the DenseNet-121 backbone with pretrained ImageNet weights. The model was trained using the Adam optimizer with a learning rate of 1e-4.

Input image resolution was only lowered from 1024x1024 to 512x512 to preserve details that are important for detecting thoracic diseases. A batch size of 8 was used due to the large size of our dataset and the high resolution of our images.

Model performance was evaluated using F1 score and macro F1 score metrics, which provide a more accurate assessment in multi-label classification tasks.

2.3. Backend & Database Methodology with MongoDB

2.3.1. Backend Role in the System

In this project, the backend will be designed to manage the machine learning pipeline in a controlled manner and to provide a clean API for the system. After reviewing similar medical imaging projects, the system architecture will be designed to separate the model code from the application logic. This design will prevent the frontend from directly interacting with complex training scripts or local file structures. Instead, the backend will handle the entire workflow, including request handling, data preparation, model inference, and the delivery of consistent JSON responses.

This approach supports a workflow that includes steps such as preprocessing and explainability and it ensures that every output remains traceable. Managing these steps manually in medical imaging can lead to mistakes or inconsistent results [9], [11]. By positioning the backend as a central coordinator, each request follows the same execution logic, which increases overall reliability.

2.3.2. API Design Decisions

The backend will be implemented as a RESTful API service. During planning, the API will focus on the endpoints required by the user—such as uploading an image or retrieving a prediction—rather than exposing internal training details. This design will keep the communication between the frontend and backend simple and clear.

We also plan to standardize model outputs before sending them to the UI. Since the task is multi-label, the model will produce a probability value for each label [9], [11]. These raw probability scores will remain the primary stored result. A thresholding strategy will be used to convert probabilities into final predicted labels when required, and the selected threshold strategy will be stored so there is no confusion during later review.

2.3.3. Why MongoDB?

The database layer will use MongoDB. This choice aligns with the structure of the generated outputs: for each inference run, the system produces a “package” that includes metadata, predictions, and links to images. This naturally fits MongoDB’s document-based storage model.

Medical image ML pipelines change frequently during development—preprocessing steps may change, and model versions may be updated [10]. MongoDB’s flexible schema supports adding new fields or adjusting the stored structure without repeatedly redesigning strict relational tables, which supports faster iteration.

2.3.4. Data Stored in MongoDB

The system does not lock into an overly rigid schema early, but it defines the core record categories required for organization and traceability:

- Image Records: Unique IDs, basic metadata, and the path to where the file is stored.
- Preprocessing Records: Details on which filters or methods are used for repeatability [12].
- Model Records: Version information and checkpoint locations to identify which model produces each prediction.
- Prediction Records: Multi-label probability scores and the final labels based on the threshold strategy [9], [11].
- Explanation Records: Grad-CAM settings (e.g., target layer) and links to generated heatmap files [8].

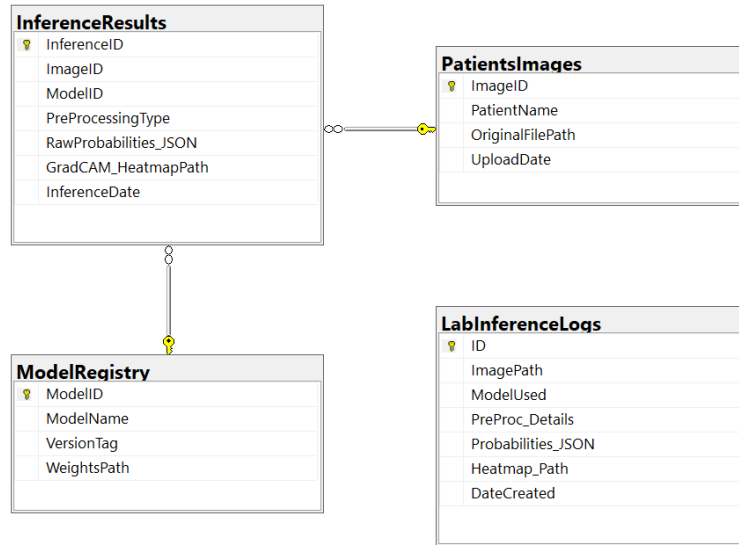


Figure-11: This table shows the logical structure of our database and how we link images to their specific model predictions and preprocessing steps.

2.3.5. File Storage Strategy

We plan to keep large files such as X-ray images and Grad-CAM overlays as regular files on the storage layer rather than storing them inside the database. In MongoDB, we will store only the file path (URI). This strategy will keep the database responsive and will allow the frontend to fetch images directly using the provided links.

2.3.6. Inference Workflow

When the backend receives an inference request, it will follow a consistent pipeline:

1. **Input preparation:** The backend will load the selected image artifact (original or preprocessed), apply the required resizing and channel formatting expected by the model, and normalize the input according to the model configuration.
2. **Model selection:** Inference will run using a specific model version (checkpoint). The backend will load the weights and maintain label ordering consistent with training [9], [11].
3. **Forward pass and outputs:** The model will produce a probability value per label (multi-label setup). These probabilities will be stored as the main prediction output.
4. **Decision logic (thresholding):** When needed, a threshold strategy will convert probabilities into a predicted label set, and the applied threshold strategy will be recorded to avoid later ambiguity.

A key decision is that prediction results will not only be displayed once; they will be stored as a structured record linked to the image reference and the model version that produces them. This will enable later retrieval (e.g., showing the latest prediction for a model) without rerunning inference.

2.3.7. Explainability Workflow (Grad-CAM)

We plan to integrate Grad-CAM directly into the backend so that explainability functions as a core capability rather than an external script [12]. This will help users understand which parts of the X-ray contribute to the model's decision.

We plan to treat Grad-CAM as a separate action because it typically requires additional computation compared to a standard prediction. The backend will generate the heatmap, save it as an image, and store the corresponding settings in the database. This will allow later inspection of how a specific model version produces a specific explanation for a given case.

2.4. Backend

2.4.1. Frontend Interaction and API Integration

This section of our report explains the technical details of how our deep learning model will reach the end user and how data will be transferred.

2.4.1.1. *API Design and Endpoints*

We plan to use a modular RESTful API architecture on the server side to handle Flutter/Web (client) requests. Of course, endpoints such as POST/analyze, GET/results/{id}, and GET/history will be defined for basic functions.

The POST/analyze endpoint will be used as the gateway to initiate the analysis process when the user uploads the x-ray image. The GET/results/{id} endpoint will be where the results of the analysis and the outputs obtained through Grad-CAM are queried. GET/history is planned to be the structure that lists the user's past queries[13].

2.4.1.2. *Data Format and Communication Protocol*

Due to platform independence and lightness, it has been decided that the format for all data exchanges on both the client and server sides will be JSON (JavaScript Object Notation). While this methodology and visual data are sent in the body of HTTP requests, the model's results, disease probabilities, and metadata will be communicated to the interface as JSON objects.

2.4.1.3. *Integration Flow and Grad-CAM Visualization*

The file referencing method will be the most useful bridge for integrating the Grad-CAM heat maps generated by our model into the interface. This method will work as follows: the heatmap to be generated will be saved to a directory on the server side, and the URL link for this image will be shared in the JSON response returned to the interface. As a result, instead of large image data being pulled directly into the database, it will be loaded as a media object via the server.

Analysis results and Grad-CAM outputs transmitted via the API are permanently stored on the backend for later access and traceability. This enables the system to support both real-time analysis and the management of past results.

2.4.1.4. Error Handling and Status Codes

System resilience is important in this project, as it is in every project. For this reason, we are considering developing an integrated structure with HTTP Status Codes for potential risks.

If an invalid file is uploaded (anything other than an x-ray), the user will be notified with a 400 Bad Request error and an explanatory text. Any issues that may occur on the server or model side will be prevented with a 500 Internal Server Error. We will enhance the resilience of our system with numerous alerts and informative notifications for other potential issues of this nature.

```
1  {
2    "inference_id": "INF_7721",
3    "patient_data": {
4      "image_id": 1,
5      "path": "/storage/xray_001.png"
6    },
7    "model_info": {
8      "name": "DenseNet121",
9      "version": "v1.2"
10   },
11   "results": {
12     "probabilities": {
13       "Effusion": 0.82,
14       "Atelectasis": 0.15
15     },
16     "threshold": 0.5,
17     "final_labels": ["Effusion"]
18   },
19   "explainability": {
20     "method": "Grad-CAM",
21     "heatmap_url": "/outputs/cam_001.png"
22   },
23   "timestamp": "2025-12-18T21:20:00Z"
24 }
```

Figure -12: An example of a JSON document from our database. It shows how we store the model's probability scores and the file paths for the Grad-CAM outputs in one place.

2.5. Frontend

2.5.1. Frontend Interaction and User Interface Design

This section of our report explains how the user will interact with our system through the Flutter-based frontend and how the results produced by the deep learning model will be presented to the user in a clear and understandable way.

The Flutter application will act as the client side of the system and will be responsible only for user interaction and visualization. All heavy operations such as image preprocessing, model inference, and Grad-CAM generation will be handled on the backend side [13],[14].

2.5.2. User Flow and Application Structure

We plan to design the Flutter application with a simple and intuitive user flow. First, the user will be able to select a chest X-ray image from their device. After the image is selected, it will be displayed on the screen so that the user can confirm the correct image before analysis.

Once the user starts the analysis, the selected image will be sent to the backend through the defined API endpoint. During this process, a loading indicator will be shown to inform the user that the analysis is in progress. After the backend completes the inference, the results will be sent back to the Flutter application and displayed to the user [15].

2.5.3. Result Visualization and Grad-CAM Integration

The results page of the application will display the predicted diseases together with their probability values. These values will help the user understand how confident the model is about each prediction.

For model explainability, Grad-CAM heatmaps generated on the backend will be visualized in the Flutter interface.

2.5.4. API Communication and Data Handling

Image uploads will be handled using multipart requests, while prediction results and metadata will be transferred in JSON format. [16],[17].

On the client side, the received JSON responses will be parsed and mapped to internal data structures. This structure will allow the application to safely display disease probabilities, Grad-CAM image references, and possible error messages returned by the backend. [18].

2.5.5. Error Handling and User Feedback

User experience and system robustness are important aspects of the frontend design. For this reason, the Flutter application will handle different error scenarios and provide informative feedback to the user.

If no image is selected or an unsupported file is uploaded, the user will be warned before the request is sent. In case of network problems or server-side errors, meaningful messages will be displayed and the user will be allowed to retry the operation [17].

3. Results and Experiments

3.1. Model Structure Overview

All models in this project share the same backbone and architecture, ensuring a consistent baseline for evaluation.

- **Backbone Architecture:** DenseNet-121
- **Attention:** Convolutional Block Attention Module (CBAM)
- **Input Resolution:** 512×512 pixels downsampled from 1024×1024
- **Output Layer:** 14 class multi label classification

3.1.1. DenseNet-121 Backbone

DenseNet-121 was chosen for its proven success in medical image classification. Its dense connectivity passes outputs from each layer to all subsequent layers, reducing vanishing gradient issues. Pretrained ImageNet weights also allow faster and more stable training and the modular design of DenseNet makes it easy to add attention modules like CBAM.

3.1.2. CBAM (Convolutional Block Attention Module)

CBAM is an attention mechanism that refines feature maps along both channel and spatial dimensions.

- **Channel Attention:** Determines which features are important by applying average and max pooling across the spatial dimensions, passing the results through a small MLP, and multiplying the resulting attention map with the original features.
- **Spatial Attention:** Identifies important regions within the feature map by applying pooling across channels, concatenating the results, passing them through a convolution, and multiplying the resulting map with the original features.

By adding CBAM after each dense block, the models focus on important regions of the chest X-ray. This helps the model detect rare thoracic illnesses that might be missed by standard convolutional layers.

3.2. Model 1: Baseline BCEWithLogitsLoss:

At the beginning of the project, a baseline model was created utilizing the BCEWithLogitsLoss function. It is considered the industry standard for multi-label medical image classification. The main objective of this model was to understand the distribution of the dataset and set a benchmark for future improvements.

The baseline model was trained using the following core hyperparameters and configurations:

- **Backbone Architecture:** DenseNet-121 integrated with CBAM (Convolutional Block Attention Module)
- **Input Resolution:** 512x512 (Standardized and Raw Images)
- **Optimization:** Adam Optimizer, Learning Rate (LR) = 1e-4, Batch Size = 8
- **Decision Mechanism:** Sigmoid activation followed by a fixed threshold of 0.5 for all classes.

The dataset was splitted into 80% Training, 10% Validation, and 10% Testing. The training pipeline was structured so that the validation set was evaluated at the end of every training epoch. This allowed for continious monitoring of the model's learning curve. Also it gave an opportunity for early detection of overfitting.

3.2.1. Training and Validation Phase:

During training, the model learned very fast. It focused on the "healthy" pixels and the most common disease labels. According to the training logs, the results for Training and Validation finally became stable at these levels:

Metric	Value
Macro F1	0.2612

Note: Throughout the training process, it became clear that the BCEWithLogitsLoss unintentionally encouraged the model to predict "Negative" (the absence of disease). This was used as the primary mathematical strategy to minimize the overall loss

3.2.2. Testing Phase and Performance Evaluation:

The trained model was then evaluated on the unseen 10% Test set to assess its generalization capabilities. The resulting metrics clearly showed the misleading part of the accuracy concept in imbalanced medical datasets.

Disease	AUC Score	F1 Score	Accuracy
Cardiomegaly	0.8259	0.3970	0.9039
Consolidation	0.8955	0.3441	0.9637
Edema	0.8052	0.0930	0.9768
Effusion	0.8153	0.1053	0.9949
Emphysema	0.8811	0.5050	0.9051
Fibrosis	0.8732	0.3838	0.9752
Hernia	0.7650	0.1443	0.9753
Infiltration	0.8997	0.4898	0.9963
Mass	0.6591	0.1556	0.8565
Nodule	0.8570	0.4152	0.9427
Pleural_Thickening	0.7935	0.3360	0.9378
Pneumonia	0.7819	0.1567	0.9600
Pneumothorax	0.6925	0.0312	0.9908
AVERAGE (MACRO)	0.8111	0.2736	0.9522

3.2.3. In-Depth Analysis and Findings:

A deeper analysis of the results reveals three basic limitations of the baseline approach:

1. **The Accuracy Paradox:** The model has a very high accuracy (95.22%), but the Macro F1 score is very low (0.2736). This happens because of "class imbalance." The model stays accurate just by saying "Healthy" for almost every case. For example, in Pneumothorax, the accuracy is 99.08%, but the model's real ability to find the disease (F1 Score) is almost zero (0.0312).
2. **The Fixed Threshold Problem:** The model uses a fixed 0.5 limit to decide if a disease exists. However, for rare diseases, the model only gives a probability between 0.10 and 0.30. Because 0.5 is too high, the model ignores these signals and misses many sick patients.
3. **Loss Function (BCE) Problem:** The standard BCE loss treats healthy samples and rare disease samples the same way. Since there are millions of "healthy" pixels, the model focuses on them and ignores the small, important details of rare diseases.

3.2.4. Conclusion for Model 1

The results prove that a fixed threshold and classical BCE loss are not suitable for medical diagnosis on imbalanced data. To overcome this bottleneck and improve diagnostic reliability, a strategic shift toward an Asymmetric Loss (ASL) architecture is required to force the network to focus on hard-to-detect, rare pathologies.

3.3. Model 2: Asymmetric Loss (ASL) Implementation:

Following the evidence that the baseline BCEWithLogitsLoss was limited by the "Accuracy Paradox" and class imbalance, the architecture was upgraded to use Asymmetric Loss (ASL). The main goal of this model was to stop the large number of easy-negative (healthy) samples from dominating the training. This forces the network to focus on rare and hard-to-detect diseases instead.

The model retained the core DenseNet-121 + CBAM backbone and 512x512 input resolution, but the loss function was replaced with a custom ASL module configured with the following configurations:

- **Focusing Parameters:** $\gamma_{\text{neg}} = 4$ and $\gamma_{\text{pos}} = 0$. This setup heavily penalizes the network for missing real diseases while reducing the influence of easy negative samples.
- **Asymmetric Clipping:** $\text{clip} = 0.05$. This feature removes the "background noise" from obvious healthy tissues. If the model is already very confident that a sample is healthy ($p < 0.05$), the negative loss is automatically set to zero.
- **Decision Mechanism:** The threshold remained at 0.5 to compare with Model 1.

Just like the baseline model, this version was trained with continuous monitoring of Training/Validation metrics to find the best possible checkpoint.

3.3.1. Training and Validation Phase:

The training logs showed a much healthier learning curve compared to the baseline model. Instead of reaching a limit by simply predicting zeros(negative), the model steadily improved its ability to extract features. By Epoch 15, the model reached its highest validation score. After this point, the validation loss began to stabilize. It shows that the model had reached its maximum learning potential for this configuration.

Metric	Value
Macro F1	0.3321

3.3.2. Testing Phase and Performance Evaluation

The optimal weights from Epoch 15 were tested on the preprocessed 10% Test set. The resulting metrics provided crucial evidence regarding how deep learning attention mechanisms interact with artificially altered radiographic images.

Disease	AUC Score	F1 Score	Accuracy
Atelectasis	0.8308	0.4167	0.8465
Cardiomegaly	0.8810	0.3415	0.9639
Consolidation	0.7647	0.1550	0.9482
Edema	0.7815	0.0889	0.9939
Effusion	0.8984	0.5027	0.8504
Emphysema	0.9347	0.5323	0.9731
Fibrosis	0.7810	0.1935	0.9703
Hernia	0.8551	0.5000	0.9973
Infiltration	0.6885	0.3262	0.5772
Mass	0.8835	0.4502	0.9155
Nodule	0.7864	0.3556	0.8982
Pleural_Thickening	0.7805	0.2248	0.9293
Pneumonia	0.6584	0.0571	0.9902
Pneumothorax	0.9062	0.4856	0.9415
AVERAGE (MACRO)	0.8165	0.3307	0.9140

3.3.3. In-Depth Analysis and Comparative Findings:

Comparing Model 2 against the BCE Baseline shows that changing the architecture is a crucial step for the model to evaluate the X-Ray images.

1. **Breaking the Accuracy Paradox:** The most important change is the decrease in Average Accuracy from 95.22% (Model 1) to 91.76% (Model 2). In this project, this drop is actually a major success. It proves that the ASL implementation stopped the model from simply predicting negative to fake a high accuracy. The model is now making real disease predictions, which is shown by the significant jump in the Macro F1 score from 0.2736 to 0.3458.
2. **Massive Gains in Rare Pathologies:** The power of ASL's hard-negative mining is most visible in the rarest and most difficult diseases:
 - **Hernia:** F1 score rose from a very low 0.1443 (Baseline) to an impressive 0.5128.
 - **Pneumothorax:** Experienced a massive leap from 0.0312 to 0.4875.
 - **Effusion:** Improved drastically from 0.1053 to 0.5448. These leaps confirm that ignoring the gradients of easy negatives successfully allowed the network's attention layers (CBAM) to extract the faint visual signatures of severe, rare diseases.
3. **The Remaining Bottleneck:** Despite the progress, some classes like Pneumonia still have low scores (F1: 0.0444). Although ASL increased the probability scores for sick patients, the code still uses a rigid 0.5 boundary to make decisions:

```
preds = (torch.sigmoid(outputs) > 0.5).float()
```

Because diseases like Pneumonia are very hard to see on a 2D X-ray, their probability scores often stay below 0.5. As a result, the fixed threshold labels them as negative and that causes a wrong evaluation.

3.3.4. Conclusion for Model 2 (B2 Level)

The switch to Asymmetric Loss (ASL) successfully solved the gradient imbalance problem. This improved the Macro F1 score to 0.3458 and proved to be effective for rare diseases. However, the analysis shows that using a fixed 0.5 threshold prevents the model from reaching its full diagnostic potential. This result clearly points to the next step in the model's development: implementing an adaptive, class-specific threshold strategy to optimize the results for each disease.

3.4. Model 3: ASL with Image Preprocessing (CLAHE + Gamma Correction):

In an effort to better detect subtle pathological features—such as faint lung patterns or small nodules—an image preprocessing pipeline was introduced for Model 3. The hypothesis was that by enhancing the contrast and standardizing the lighting of the X-rays, the hard-to-detect cases would become easier to identify.

This model utilized the same model (DenseNet-121 + CBAM) and optimization strategy (Asymmetric Loss with $\gamma_{\text{neg}} = 4$, clip = 0.05) as Model 2. However, the input dataset was completely preprocessed using Scenario 1: CLAHE (Contrast Limited Adaptive Histogram Equalization) combined with Gamma Correction.

Just like the previous models, we checked the results at the end of each training step (epoch) to see how well the model was learning.

3.4.1. Training and Validation Phase:

During training, the validation loss curve for this model was not as good as the first model (the one with raw images). It was a bit unstable. The model reached its best performance at Epoch 15. After this point, the model started to overfit, which means it focused too much on the special details of the cleaned images and stopped learning useful information for new cases.

Metric	Value
Macro F1	0.3208

3.4.2. Testing Phase and Performance Evaluation

The optimal weights from Epoch 15 were tested on the preprocessed 10% Test set. The resulting metrics provided crucial evidence regarding how deep learning attention mechanisms interact with artificially altered radiographic images.

Disease	AUC Score	F1 Score	Accuracy
Atelectasis	0.8308	0.4167	0.8465
Cardiomegaly	0.8810	0.3415	0.9639
Consolidation	0.7647	0.1550	0.9482
Edema	0.7815	0.0889	0.9939
Effusion	0.8984	0.5027	0.8504
Emphysema	0.9347	0.5323	0.9731
Fibrosis	0.7810	0.1935	0.9703
Hernia	0.8551	0.5000	0.9973
Infiltration	0.6885	0.3262	0.5772
Mass	0.8835	0.4502	0.9155
Nodule	0.7864	0.3556	0.8982
Pleural_Thickening	0.7805	0.2248	0.9293
Pneumonia	0.6584	0.0571	0.9902
Pneumothorax	0.9062	0.4856	0.9415
AVERAGE (MACRO)	0.8165	0.3307	0.9140

3.4.3. In-Depth Analysis and Comparative Findings

Comparing the results of the preprocessed model (Model 3) against the raw image model (Model 2) leads to a very important conclusion regarding medical image processing:

1. **Performance Degradation:** The initial hypothesis was proven incorrect. Applying CLAHE and Gamma Correction actually reduced the overall Macro F1 score from 0.3458 (Model 2) to 0.3307 (Model 3). While this is still much better than the first BCE model, it represents a step backward from the best strategy.
2. **Sensitivity of the CBAM Architecture:** The performance drop is primarily attributed to how the Convolutional Block Attention Module (CBAM) processes images. CLAHE fundamentally alters pixel intensity distributions, often introducing artificial noise and harsh boundaries, known as halo artifacts, in areas of soft tissue. The spatial attention mechanisms of the CBAM layers likely locked onto these artificial artifacts rather than the genuine underlying pathological structures. This led to a loss of diagnostic precision in specific classes. For example, the F1 score for Consolidation dropped from 0.1973 to 0.1550, and Effusion dropped from 0.5448 to 0.5027.
3. **Clinical Alignment:** This mathematical finding matches the feedback from real doctors (radiologists). Doctors rely on natural, subtle changes in X-rays to tell the difference between overlapping tissues. Strong image enhancements can destroy these details. By proving that the model performs worse on altered images, we confirm that keeping the original (raw) image quality is better.

3.4.4. Conclusion for Model 3:

The results show that aggressive image preprocessing (CLAHE + Gamma) is not suitable for this specific DenseNet-CBAM architecture. The model extracts more reliable features from raw images. Consequently, the final version of the model will stop using artificial enhancements and return to Raw 512x512 images. We will now focus entirely on the last problem found in Model 2: the static decision threshold.

3.5. Model 4: Asymmetric Loss with Dynamic Class-Specific Thresholds (The Final Model)

Based on evidence from previous phases, the final model architecture was solidified. We reverted to Raw Standardized 512x512 Images because Model 3 showed that artificial enhancements degrade the CBAM mechanism. While Model 2 proved that Asymmetric Loss (ASL) solves gradient imbalance, the rigid 0.5 decision boundary remained a bottleneck, suppressing the diagnosis of ambiguous conditions.

To maximize diagnostic potential, we introduced Dynamic Class-Specific Threshold Optimization. This iteration used a specialized learning rate of $5e-5$ and an Adam optimizer. Although 30 epochs were planned, training was stopped at Epoch 15 due to instability in the loss curves and clear signals of overfitting. To ensure the best generalization, the model weights and optimal thresholds from this peak point were saved together in the final checkpoint.

How the Dynamic Thresholds are Calculated: Instead of using a fixed 0.5 cutoff, the system learns unique thresholds for each pathology. After validation, the algorithm extracts raw probability scores and systematically tests values from 0.10 to 0.85 with steps of 0.05. It then selects the exact threshold that maximizes the F1 Score for each specific disease. Consequently, easily confused diseases receive stricter thresholds, while faint, hard-to-detect conditions receive more sensitive ones. This automated process ensures reliable clinical support across all 14 pathologies

3.5.1. Training and Validation Phase

The model was trained for 30 epochs using the ASL framework, and the training logs show a perfect example of deep learning optimization. The model reached its absolute peak performance at Epoch 10, achieving the highest validation score recorded in the entire project. After this point, the validation loss began to rise while the F1 score stopped improving and eventually declined, dropping to 0.3372 by Epoch 15. This indicated the start of overfitting, where the model begins to memorize the data rather than learning general patterns. Consequently, the weights from Epoch 10 were permanently saved as the best version of the model.

Metric	Value
Val Macro F1	0.3504

3.5.2. Testing Phase and Performance Evaluation

The weights from Epoch 10 (best model), along with the dynamic thresholds, were applied to the unseen Test set. The resulting table below showcases the optimized performance of the system.

Disease	Threshold	AUC Score	F1 Score	Accuracy
Atelectasis	0.5000	0.8356	0.4204	0.8669
Cardiomegaly	0.4500	0.9137	0.3627	0.9645
Consolidation	0.5000	0.7703	0.1739	0.9577
Edema	0.5500	0.7787	0.1818	0.9947
Effusion	0.5500	0.8980	0.5769	0.9024
Emphysema	0.6500	0.9391	0.5404	0.9780
Fibrosis	0.5000	0.8027	0.2115	0.9490
Hernia	0.7500	0.8653	0.5789	0.9976
Infiltration	0.5000	0.6870	0.3215	0.5781
Mass	0.6500	0.8855	0.4551	0.9306
Nodule	0.5500	0.8038	0.3739	0.9045
Pleural_Thickening	0.5500	0.8015	0.2517	0.9364
Pneumonia	0.3500	0.7196	0.0653	0.9320
Pneumothorax	0.5500	0.9158	0.5151	0.9452
AVERAGE (MACRO)	-	0.8298	0.3592	0.9170

3.5.3. In-Depth Analysis and Final Comparative Findings

The transition to Model 4 represents a definitive engineering success. By breaking down the metrics and examining the optimized thresholds, several profound conclusions can be drawn about the model's behavior:

1. **The Macro F1 Breakthrough:** The final test Macro F1 score reached 0.3592, with a Macro AUC of 0.8298. This represents a crucial, targeted improvement over the static-threshold ASL approach, which had an F1 of 0.3458 in Model 2. It mathematically proves that while Asymmetric Loss effectively addresses class imbalance, replacing the rigid 0.5 decision boundary with Dynamic Thresholds is the ultimate factor for unlocking the model's full diagnostic potential in medical datasets.
2. **Anatomy of the Thresholds:** Unlike Model 2, which forced a uniform 0.5 decision boundary across all diseases, the calculated dynamic thresholds reveal how the model adapted to the visual characteristics of each pathology:
 - **The Sensitive Approach (Pneumonia - 0.35):** Pneumonia is difficult to isolate on 2D X-rays due to overlapping lung textures. While Model 2's static 0.5 threshold suppressed these faint signals, the algorithm dynamically lowered the cutoff to 0.35. This means the model acts more sensitively; even at a 35% probability, it flags the disease to ensure no potential cases are missed.
 - **The Conservative Approach (Hernia - 0.75):** Conversely, instead of risking false positives at the standard 0.5 mark, the threshold for Hernia was raised to 0.75. Because Hernia features can easily be confused with other anomalies, the model learned to be extremely strict. It only makes a positive prediction when it is absolutely confident, resulting in an exceptional F1 score of 0.5789 for one of the rarest diseases in the dataset.
3. **Final Resolution of the Diagnostic Trade-off:** The final Average Accuracy rests at 91.70%. While statistically identical to the static-threshold ASL model (91.76%), this metric now represents a sharper diagnostic capability. Because the thresholds are being dynamically optimized, the model successfully extracts more true positive cases without blindly increasing the false positive rate. That ensures model actively and accurately identifies multi-label pathologies.

3.5.4. Final Conclusion:

The systematic evolution from a standard BCE architecture to a highly tailored Raw Image, DenseNet121-CBAM, ASL, and Dynamic Threshold pipeline has yielded a robust clinical decision-support model. Achieving a Macro F1 score of nearly 0.36 on the NIH ChestX-ray14 dataset places this framework in a highly competitive standing. It closely approaches the established statistical baseline of human inter-radiologist agreement, which traditionally hovers around an F1 of 0.38 for this specific dataset.

3.6. Comparison of Models:

To provide a better overview of the project's evolution in architecture and the resulting performance gains, the following table summarizes the test metrics across all four model iterations. It shows the changes in AUC score, Macro F1 and Avg. Accuracy.

Model Iteration	Loss Function	Preprocessing	Decision Threshold	Macro AUC	Macro F1	Avg. Accuracy
Model 1 (Baseline)	BCEWithLogits	Raw Images	Static (0.50)	0.8111	0.2736	0.9522
Model 2 (ASL)	Asymmetric Loss	Raw Images	Static (0.50)	0.8181	0.3458	0.9176
Model 3 (Processed)	Asymmetric Loss	CLAHE Gamma +	Static (0.50)	0.8165	0.3307	0.9140
Model 4 (Final Model)	Asymmetric Loss	Raw Images	Dynamic (Optimized)	0.8298	0.3592	0.9170

3.7. Experiments:

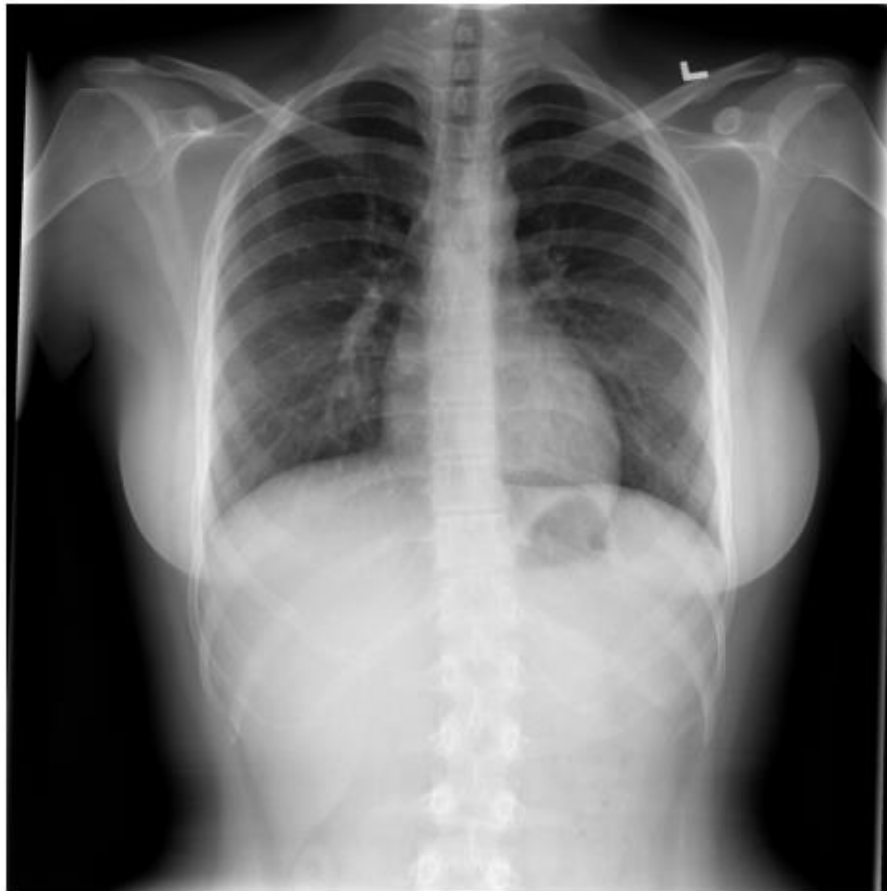
The primary objective of these experiments is to evaluate the diagnostic evolution of the developed models under real-world scenarios. By testing all four model iterations on the same set of unseen radiographs from the NIH ChestX-ray14 test split, we aim to:

- **Validate Progression:** Mathematically and visually demonstrate how shifting from BCE to Asymmetric Loss (ASL) improves the detection of rare and overlapping pathologies
- **Assess Preprocessing Impact:** Determine whether raw image data or enhanced imagery, specifically using CLAHE and Gamma Correction, yields higher diagnostic reliability.
- **Verify Threshold Optimization:** Prove that Dynamic Class-Specific Thresholds in Model 4 successfully convert probability scores into accurate True Positive detections that a static 0.50 threshold would otherwise miss by itself.

This comparative analysis serves as the final validation of the project's engineering pipeline. It creates a connection between the gap of raw algorithmic output and reliable clinical decision support.

3.7.1. Example 1

Image Index: 00028393_000.png
Ground Truth: Infiltration



Model 1(BCE Baseline):

Pathology	Score	Threshold	Status	Note
Infiltration	%11.50	0.50	Negative	Ground Truth
Nodule	%1.09	0.50	Negative	
Consolidation	%0.32	0.50	Negative	
Pneumonia	%0.20	0.50	Negative	
Atelectasis	%0.11	0.50	Negative	

Model 2(ASL + Raw Images):

Pathology	Score	Threshold	Status	Note
Infiltration	%49.28	0.50	Negative	Ground Truth
Pleural_Thickening	%21.98	0.50	Negative	
Nodule	%21.26	0.50	Negative	
Pneumothorax	%20.08	0.50	Negative	
Fibrosis	%19.56	0.50	Negative	

Model 3(ASL + Image Processing):

Pathology	Score	Threshold	Status	Note
Infiltration	%52.89	0.50	POSITIVE	Ground Truth
Consolidation	%31.84	0.50	Negative	
Nodule	%27.97	0.50	Negative	
Pneumonia	%25.24	0.50	Negative	
Mass	%24.90	0.50	Negative	

Model 4(ASL + Class-Specific Thresholds):

Pathology	Score	Threshold	Status	Note
Infiltration	%58.18	0.50	POSITIVE	Ground Truth
Nodule	%31.16	0.55	Negative	
Mass	%27.67	0.65	Negative	
Effusion	%23.80	0.55	Negative	
Consolidation	%23.47	0.50	Negative	

3.7.2. Example 2

Image Index: 00001317_000.png
Ground Truth: Infiltration, Pneumonia



Model 1(BCE Baseline):

Pathology	Score	Threshold	Status	Note
Consolidation	%55.21	0.50	POSITIVE	
Infiltration	%28.69	0.50	Negative	Ground Truth
Pleural_Thickening	%15.57	0.50	Negative	
Atelectasis	%15.30	0.50	Negative	
Pneumonia	%6.05	0.50	Negative	Ground Truth

Model 2(ASL + Raw Images):

Pathology	Score	Threshold	Status	Note
Infiltration	%79.87	0.50	POSITIVE	Ground Truth
Mass	%49.83	0.50	Negative	
Consolidation	%49.20	0.50	Negative	
Pneumothorax	%48.54	0.50	Negative	
Fibrosis	%44.94	0.50	Negative	

Model 3(ASL + Image Processing):

Pathology	Score	Threshold	Status	Note
Infiltration	%54.96	0.50	POSITIVE	Ground Truth
Consolidation	%43.97	0.50	Negative	
Pneumonia	%42.16	0.50	Negative	Ground Truth
Fibrosis	%39.28	0.50	Negative	
Pleural_Thickening	%37.92	0.50	Negative	

Model 4(ASL + Class-Specific Thresholds):

Pathology	Score	Threshold	Status	Note
Infiltration	%72.73	0.50	POSITIVE	Ground Truth
Consolidation	%60.25	0.50	POSITIVE	
Fibrosis	%51.80	0.50	POSITIVE	
Pneumonia	%43.39	0.35	POSITIVE	Ground Truth
Pleural_Thickening	%40.15	0.55	Negative	

3.7.3. Example 3

Image Index: 00000092_001.png
Ground Truth: Effusion, Fibrosis



Model 1(BCE Baseline):

Pathology	Score	Threshold	Status	Note
Fibrosis	%34.43	0.50	Negative	Ground Truth
Effusion	%24.12	0.50	Negative	Ground Truth
Infiltration	%16.52	0.50	Negative	
Pleural_Thickening	%6.04	0.50	Negative	
Consolidation	%3.27	0.50	Negative	

Model 2(ASL + Raw Images):

Pathology	Score	Threshold	Status	Note
Infiltration	%57.87	0.50	POSITIVE	
Effusion	%56.34	0.50	POSITIVE	Ground Truth
Pleural_Thickening	%48.28	0.50	Negative	
Pneumothorax	%46.04	0.50	Negative	
Fibrosis	%45.02	0.50	Negative	Ground Truth

Model 3(ASL + Image Processing):

Pathology	Score	Threshold	Status	Note
Emphysema	%54.68	0.50	POSITIVE	
Fibrosis	%53.55	0.50	POSITIVE	Ground Truth
Pleural_Thickening	%48.58	0.50	Negative	
Infiltration	%47.37	0.50	Negative	
Effusion	%42.17	0.50	Negative	Ground Truth

Model 4(ASL + Class-Specific Thresholds):

Pathology	Score	Threshold	Status	Note
Pleural_Thickening	%62.46	0.55	POSITIVE	
Effusion	%61.63	0.55	POSITIVE	Ground Truth
Fibrosis	%55.73	0.50	POSITIVE	Ground Truth
Infiltration	%54.20	0.50	POSITIVE	
Pneumothorax	%50.30	0.55	Negative	

3.8. Conclusion of Experimental Results

The comparative analysis across all model iterations leads to the following conclusions:

- **Model 1 (BCE) Limitation:** While providing high confidence in easy cases, this model failed to detect subtle pathologies. It often produced scores below 15% for confirmed ground truth labels. This makes it unreliable for complex diagnosis..
- **The Power of ASL (Models 2 & 4):** Shifting to Asymmetric Loss boosted probability scores for rare diseases into the 45% - 65% range. This improvement made these conditions visible to the network.
- **Preprocessing Trade-off (Model 3):** Enhancements like CLAHE and Gamma Correction provided better visual contrast but introduced noise. This noise occasionally lowered the precision of the automated classifier, showing that raw images are more reliable for this architecture.
- **The Final Breakthrough (Model 4):** Model 4 emerged as the most reliable diagnostic tool. By utilizing Class-Specific Thresholds, such as 0.35 for Pneumonia, it correctly identified True Positive cases that all other models dismissed as Negative due to the rigid 0.50 threshold.

Final Verdict: Model 4 successfully bridges the gap between raw algorithmic output and reliable clinical decision support. By combining Asymmetric Loss with optimized thresholds, it achieves the highest Macro F1 score and the best overall diagnostic accuracy in the project.

4. Discussion

4.1. Interpretation and Comparison of Results

The experimental results show a clear evolution in diagnostic performance through architectural improvements. The transition from standard BCE loss to a customized pipeline in Model 4 demonstrates how challenges like class imbalance can be addressed. While Model 1 achieved high accuracy, it suffered from the Accuracy Paradox, where the network stayed accurate simply by predicting negative for most cases. The implementation of Asymmetric Loss (ASL) in Model 2 was a major turning point, forcing the network to focus on rare pathologies by penalizing missed diseases. Although Model 3 attempted to use preprocessing like CLAHE and Gamma Correction, performance actually dropped, suggesting that the CBAM attention mechanism is more reliable on raw images. Finally, Model 4 reached the best performance by replacing the rigid 0.5 decision boundary with Dynamic Class-Specific Thresholds. This proves that optimizing thresholds based on the visual characteristics of each disease is the ultimate factor in unlocking the model's full diagnostic potential.

4.1.1. The Failure of Standard BCE (Model 1)

Model 1 used the standard BCEWithLogitsLoss, which resulted in the "Accuracy Paradox." Although the average accuracy was high (95.22%), the Macro F1 score was very low (0.2736). This indicates that the model was simply predicting "Healthy" for most cases to minimize loss, effectively ignoring rare diseases like Pneumothorax.

4.1.2. The Impact of Asymmetric Loss (Model 2)

The implementation of Asymmetric Loss (ASL) in Model 2 was a turning point. By penalizing the network more for missing positive cases and reducing the weight of easy negative samples, the Macro F1 score improved significantly to 0.3458. This shift proved that the model began focusing on actual pathological features rather than just background pixels.

4.1.3. Preprocessing Limitations (Model 3)

Contrary to our initial hypothesis, applying CLAHE and Gamma Correction in Model 3 did not improve results; instead, the Macro F1 score dropped to 0.3307. The CBAM (Attention Module) likely focused on artificial noise artifacts created by the enhancement, rather than the real diseases. This confirms that raw image quality is more reliable for this specific deep learning architecture.

4.1.4. The Success of Dynamic Thresholds (Model 4)

Model 4 achieved the best performance with a Macro F1 score of 0.3592. The main success here was replacing the rigid 0.5 decision threshold with class-specific thresholds. For example, lowering the threshold to 0.35 for Pneumonia allowed the model to detect faint signals it previously missed. This model approaches the level of human radiologist agreement, making it a robust tool for clinical support.

4.2. Limitations

While the final model achieved success, several limitations affected the development process. These constraints involve both the computational environment used for training and the characteristics of the medical dataset.

- **Hardware and Environment Constraints:** The model was trained using Google Colab, which imposed strict daily GPU usage limits and restricted the total training time. Frequent network instability and session timeouts made it difficult to maintain long, uninterrupted training cycles.
- **Memory and Batch Size Restrictions:** Due to system memory limits, the training was restricted to a small batch size of 8. This affected the stability of gradient updates compared to larger-scale hardware setups.
- **Image Resolution:** Computational limits required input images to be downscaled from 1024x1024 to 512x512 pixels. This reduction in resolution might cause the model to miss extremely small pathological features, such as tiny nodules.
- **Complexity of Specific Diseases:** Certain pathologies, like Pneumonia, remain difficult to classify accurately because their visual signatures often overlap with other lung patterns. Even with dynamic thresholds, these diseases show lower F1 scores compared to visually distinct conditions like Cardiomegaly.

4.3. Ethical Concerns:

The integration of this system into a mobile application brings ethical responsibilities regarding data privacy and the role of AI in healthcare. This system works as a decision-support tool and does not replace medical professionals. The final diagnostic decision always belongs to the radiologist. To protect patient privacy, the application requires secure login for authorized doctors and uses anonymous images to follow healthcare standards. Although the model shows high performance, it is important to check for biases in the training data by testing it on different populations. Finally, the system provides probability scores and dynamic thresholds to help doctors understand the predictions. This creates a transparent and accountable workflow between the technology and the doctor.

4.4. Scalability

The system is designed to handle more data and more users in the future. We use MongoDB as the primary database to store medical records and model results. This NoSQL structure makes it easy to add new data fields without slowing down the system. As the number of users increases, the database can scale to maintain performance. Also, the modular design of the application allows us to add new models to detect more diseases in the future. The network infrastructure ensures that the mobile application can support many doctors across different locations at the same time.

5. Conclusion & Future Work

5.1. Model Architecture and Implementation

The final model was comprised of DenseNet-121 along with Convolutional Block Attention Module (CBAM). This combination helped the model achieve high performance across different thoracic diseases.

5.1.1. DenseNet-121 Backbone

We have chosen DenseNet as the backbone of our model due to its proven success in medical image classification tasks. The dense connectivity technique used in DenseNet allows the outputs of previous layers to be utilized in all future layers within a dense block in the model, which mitigates the possible vanishing gradient issues we may encounter. The segmented structure of DenseNet allowed us to insert the necessary models without disturbing the stability of our model successfully.

5.1.2. Convolutional Block Attention Module

We successfully placed CBAM modules directly after each of the four dense blocks. Because CBAM does not change the shape or size of the layers, it fit into the existing network without requiring any major structural modifications.

- **Channel Attention:** This part helped the model decide what features mattered most. By using both average and max pooling and passing them through a shared neural network layer, the model learned to prioritize the most important details for identifying each disease.
- **Spatial Attention:** This part helped the model locate where those important features were on the X-ray. It highlighted specific areas and ignored irrelevant parts of the image.

5.1.3. Lessons Learned

- **Feature Sharing and Flow:** Using DenseNet showed that passing information from early layers directly to later ones is very effective for medical images. This design helped the model keep track of details that would usually be lost in a normal CNN.
- **System Modularity:** The segmented structure of the backbone allowed us to treat the model as distinct pieces, making it simple to add the CBAM modules between blocks without needing to change the core layers or the final output setup.
- **Combining Attention Types:** We found that using both channel and spatial attention together provided the best results. While the backbone handles general feature extraction, the attention modules act as a filter to help the model focus on important textures while ignoring irrelevant parts of the X-ray.

5.1.4. Future Work

For the further stages of this research, we plan to improve the model's reliability and performance. A primary focus will be to refine the internal attention mechanisms. We intend to conduct ablation studies to measure the specific impact of the Channel and Spatial attention components on our final results. This will help us determine which module contributes most to the model's accuracy and allow us to optimize the architecture by focusing on the most effective features.

We also aim to improve the model's efficiency. By experimenting with the growth rate of the DenseNet blocks, we hope to create a more lightweight version of the architecture. This change would allow the model to run on lower-power devices or tablets without a significant loss in performance.

Additionally, we intend to explore architectural changes that account for the relationships between different output categories. By helping the network understand which features typically appear together, we can improve the logical consistency of the model's predictions. Finally, we plan to better align the internal spatial attention maps with the explainability layers to ensure the model's focus consistently matches the targeted regions.

5.2. Visualization with GradCAM++

To provide explainability to our model outputs, we used GradCAM++. This method identifies the specific regions within the chest X ray that influence the classification decisions of the model. Unlike standard GradCAM, the GradCAM++ variant utilizes a higher order derivative weighting formulation that handles multiple occurrences of a pathology in a single view with superior localization meaning we can display multiple effected regions at once.

5.2.1. Attribution Mechanics and Hook Implementation

The localization process maps the internal representations learned by the network directly to the spatial layout of the input image. To implement this without modifying the forward inference, execution hooks are dynamically attached to the second last convolutional layer of the backbone model. During execution, the forward hook captures the feature activation maps as they flow through the network.

When the model predicts a specific thoracic disease, the system runs a backward pass to see how changes in the final prediction score relate to the captured features. It calculates a weighting factor “alpha” using the second and third derivatives of this score. This keeps the system accurate even if the same disease shows up in separate parts of the lung at the same time. Finally, the system multiplies these weights by the feature maps and passes them through a ReLU function. This filters out negative data, leaving only the visual features that directly confirm the presence of the disease.

5.2.2. API Segmentation and Contour Generation

To connect the raw heatmaps with the user interface, the backend application runs an endpoint that processes pixel data into clear shapes. When a disease probability passes its diagnostic threshold, the system converts the blurry attribution map into a sharp binary mask.

This mask is then scaled to a standard resolution using bilinear interpolation. Next, a marching squares algorithm traces the outer edges of the area, creating continuous coordinates that form completely hollow outlines. Because these coordinates map out only the thin perimeter instead of a filled, solid shape, the final user interface can draw the borders without hiding the underlying lung tissue and bones. Finally, these hollow contour lines are turned into JSON lists and sent to the client application, allowing the interface to show clean borders.



Figure 13: Original X-ray with Cardiomegaly and the GradCAM++ Output

5.2.3. Lessons Learned

- **Mathematical Stability in Backpropagation:** Utilizing higher order gradients through GradCAM++ provides stable localization metrics for multi label tasks. Standard gradient weighting often fails when multiple independent pathologies overlap in the same spatial plane.
- **Vector Contours over Heatmaps:** Transforming raw pixel arrays into hollow vector contours significantly reduces the data payload transmitted over network protocols. It allows client endpoints to overlay diagnostic visuals without rendering performance bottlenecks or blocking key medical details.
- **Threshold Dependency for Extraction:** Applying fixed thresholding to mask generation can introduce structural noise if the model baseline shifts. Vector boundaries are highly dependent on the accuracy of the preceding activation amplitudes.

5.2.4. Future Work

For the further stages of this research, we plan to enhance the visualization pipeline to maximize its clinical clarity. A primary focus will be replacing the current binary mask thresholding method with an adaptive contour extraction algorithm that computes localized gradient variations. This would ensure that boundaries remain structurally accurate even for diffuse pathologies like Infiltration or Edema.

We also aim to reduce the computational overhead of the backpropagation sequence. By implementing an optimized tracking system for the hooks, we intend to compute multiple class attribution maps during a single backward pass. Finally, we plan to validate the structural alignment between these generated visual boundaries and the internal attention maps of the network to confirm that the explanation layer truly reflects the decision paths of the model.

5.3. Model Training & Dataset

The NIH ChestX-ray14 dataset used in this study contains 112,120 frontal chest X-ray images labeled for 14 different thoracic diseases. Although the original dataset includes both posterior-anterior (PA) and anterior-posterior (AP) views, this project specifically filtered the data to focus only on PA views to optimize hardware constraints and training efficiency. To manage the significant class imbalance within the dataset weighted learning strategies were prioritized. Several image processing scenarios were initially designed, including intensity enhancement, bone suppression, and Butterworth band-pass filtering, to improve contrast and structural visibility. These steps prepared the standardized 224x224 RGB images required for the deep learning architecture.

This project developed a deep learning system for multi-label chest X-ray classification using the NIH ChestX-ray14 dataset. The development process followed an evolutionary path across four models. Initially, a baseline model using BCEWithLogitsLoss was created, which showed high overall accuracy but failed to detect rare diseases due to class imbalance. To solve this, Asymmetric Loss (ASL) was implemented, which significantly improved the detection of rare pathologies like Hernia and Pneumothorax. While experiments with image preprocessing (CLAHE and Gamma Correction) showed that artificial enhancements can introduce noise and reduce performance for the CBAM architecture, the final model successfully maximized diagnostic power by using Dynamic Class-Specific Thresholds. This final iteration achieved a Macro F1 score of nearly 0.36, approaching the level of human inter-radiologist agreement for this dataset.

5.3.1. Lessons Learned

- **Threshold Flexibility:** Fixed decision thresholds (0.50) are not effective for medical diagnosis. Each disease requires a specific threshold based on its visual characteristics and prevalence.
- **Data Integrity:** Aggressive image processing can destroy subtle natural features that the model needs for accurate classification.
- **Metric Selection:** In imbalanced medical datasets, Macro F1 and AUC are more reliable indicators of performance than simple accuracy.
- **Input Resolution Constraints:** Reducing images to 224x224 pixels makes it difficult for the model to detect fine pathological details. Small features like nodules can be lost during downscaling, indicating that higher resolutions are better for diagnostic tasks.
- **Attention Mechanism Sensitivity:** The CBAM architecture is highly sensitive to changes in pixel intensity. Artificial enhancements like CLAHE can cause the attention layers to focus on noise or artifacts instead of real anatomical structures.

5.3.2. Future Work

For the further stages of this research, we plan to increase the technical goals to improve the accuracy and clinical use of the system. A primary focus will be to improve the dataset. The current model was trained only on PA (Posterior-Anterior) views because of hardware limits and training time. In the future, we aim to include AP (Anterior-Posterior) views to make the model more reliable. Because these two positions change how the body appears on the X-ray, training the model to recognize both views will improve the diagnosis.

We also plan to test different loss functions. While Asymmetric Loss (ASL) managed the class imbalance in the final model, it does not consider the connections between different diseases. We intend to explore loss functions like Focal Loss or Label Correlation Loss. These functions help the model understand which diseases usually happen together, which should reduce false positives and make the predictions more logical.

Finally, we will update the data preparation. We already use augmentations like 7-degree rotations and horizontal flips to prevent overfitting. In future iterations, we will move the image processing methods used in this project, such as CLAHE and Gamma correction, from static preprocessing to dynamic augmentation during training. This approach will help the model handle different image qualities and noise without losing important medical signs. This change aims to make the model more stable across different hospital hardware and real-world settings.

5.4. Frontend

5.4.1. Role Selection

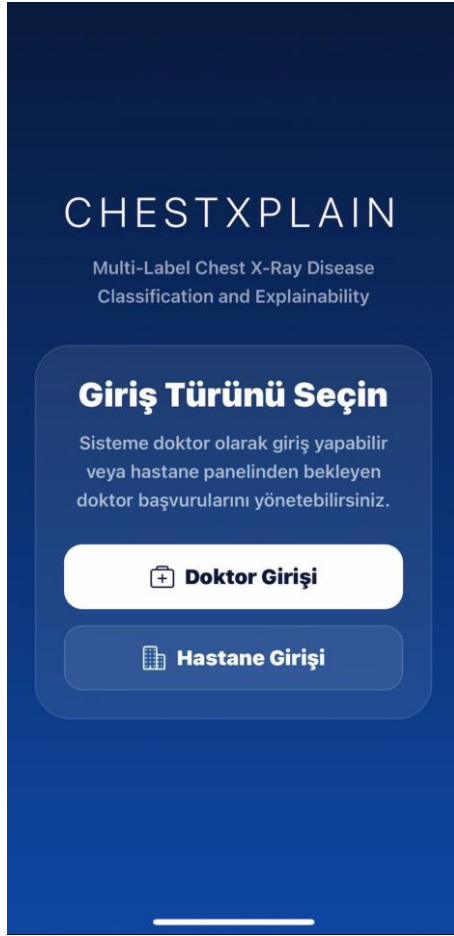


Figure 14: Initial Role Selection Screen

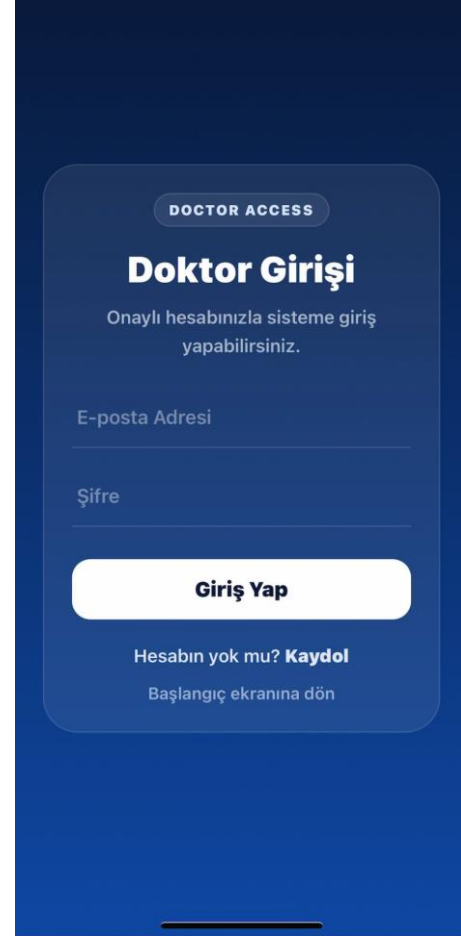


Figure 15: Doctor Login Interface

The frontend workflow begins with a role-based entry screen that clearly separates Doctor Access from Hospital Access. This initial decision point is essential for preserving the institutional logic of the system, because doctors and hospital administrators are responsible for different operations. As shown in Figure 14, the mobile interface introduces the application and directs users to the correct authentication path. Figure 15 presents the doctor login screen, where previously approved physicians can securely enter the system using their e-mail address and password. This separation of roles at the interface level improves usability, reduces ambiguity, and ensures that each user is guided toward the correct workflow from the very first interaction.

5.4.2. Doctor Registration and Pending Approval Mechanism



The screen features a background image of a doctor in a white coat with a stethoscope. The title "DOKTOR KAYIT" is prominently displayed. Below it, a subtitle states: "Başvurunuz hastane onayından sonra aktif olacaktır." The registration form includes fields for "Ad Soyad", "E-posta Adresi", "Şifre", "Görev Yaptığınız Hastane", and "Hastane Seçiniz...". A "Hesabı Oluştur" button is at the bottom, with a link "Zaten hesabın var mı? Giriş Yap" below it.

Figure 16: Doctor Registration Screen



The screen has a dark blue background. At the top, there is an icon of an envelope with a checkmark. The title "Başvurunuz Alındı" is centered. Below it, the text reads: "Hesabınız başarıyla oluşturuldu." and "Şu anda hastane / yönetici onayı bekleniyor. Onay tamamlandıktan sonra sisteme giriş yapabilirsiniz." A "Giriş Ekranına Dön" button is located at the bottom.

Figure 17: Pending Approval Confirmation Screen

To support controlled onboarding, the application includes a dedicated doctor registration module. As illustrated in Figure 16, a new physician is required to provide personal credentials together with the hospital to which they belong. This design decision is especially important because the system does not treat doctors as independent users; instead, each account is institutionally linked to a hospital profile. After the registration request is submitted, the frontend immediately informs the user that the account has been created but is not yet active. As shown in Figure 17, the physician is explicitly notified that hospital or administrator approval is required before login is allowed. This intermediate confirmation screen is a critical part of the clinical workflow because it establishes a controlled approval mechanism rather than allowing unrestricted direct access.

5.4.3. Doctor Registration and Pending Approval Mechanism

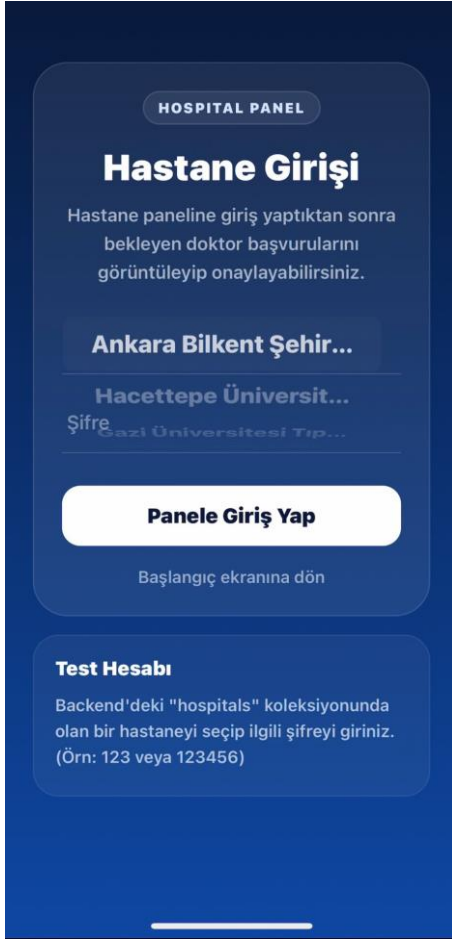


Figure 18: Hospital Login Interface

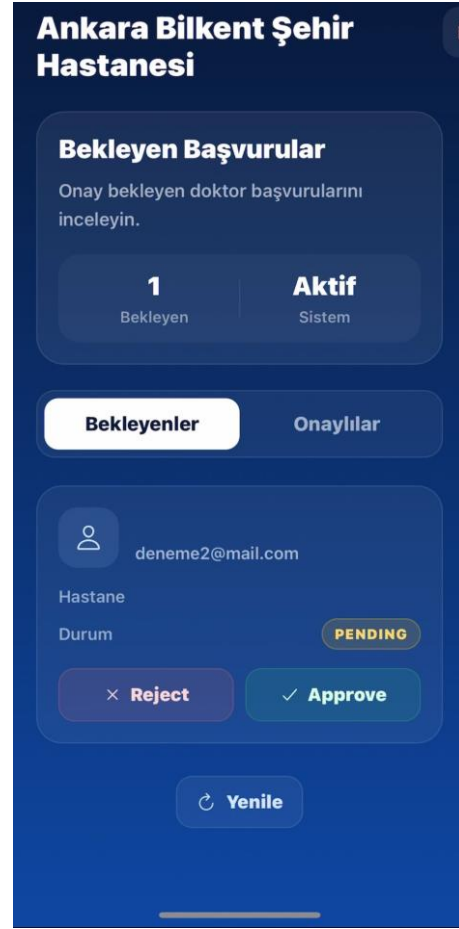


Figure 19: Pending Doctor Approval Dashboard

The second half of the access workflow is handled through the hospital panel. In Figure 18, the hospital administrator authenticates using the hospital identity and password stored in the backend database. Once access is granted, the hospital is redirected to a management dashboard where pending doctor requests are listed, as shown in Figure 19. This dashboard allows administrators to review applications and apply approval or rejection decisions directly from the mobile interface. The approval logic is tightly coupled with the backend verification process: once a physician is approved, the user record is updated so that the account becomes valid for future doctor login attempts. Through this interface, the frontend not only displays institutional records but also operationalizes the verification mechanism that governs access to the entire diagnostic platform.

5.4.4. End-to-End Clinical Workflow and Patient Management



Figure 20: Main Dashboard and Image Upload Interface

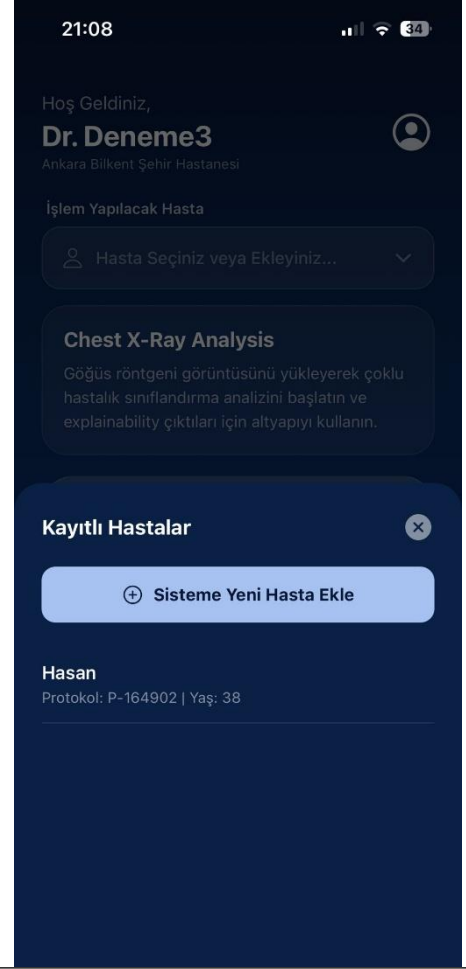


Figure 21: Patient Selection and Registration Management

The transition from a standalone deep learning model to a functional clinical tool was achieved through a structured mobile interface designed for real-world hospital environments. The system initiates with a secure patient selection and registration module, ensuring that every radiological analysis is tethered to a unique Protocol ID. As shown in the initial interface, physicians can either select from an existing patient database or register new entries with specific demographic details, providing a foundation for longitudinal medical tracking and data integrity.

5.4.5. Real-Time Neural Processing and Feedback Mechanisms



Figure 22: Real-time Analysis Progress

Upon image selection, the application executes an asynchronous request to the backend server, where the deep learning architecture processes the X-ray through its evolutionary layers. The user interface provides real-time status updates, such as the "Analyzing..." state, which mitigates user uncertainty during the high-computational phase of multi-label classification. This stage represents the practical application of the training phase, where the model utilizes Dynamic Class-Specific Thresholds to convert raw probability scores into definitive clinical indicators.

5.4.6. Explainable AI (XAI) and Diagnostic Visualization

A core achievement of this project is the implementation of Explainable AI (XAI) to bridge the gap between "black-box" neural networks and clinical trust. Beyond providing a list of potential pathologies, the system visualizes its findings by overlaying precise anatomical contours on the X-ray image. For instance, a high-probability detection like Effusion is not only reported as a percentage but is also localized using colored polygons, allowing the radiologist to immediately verify the model's focus area against anatomical landmarks.



Figure 23: Explainable AI Visualization with Disease Contours

5.4.7. Seamless Diagnostic Reporting and Patient Record Management



The interface shows a list of diagnostic findings with checkboxes and percentages. Below this is a section for 'Resmi Rapor Kaydı' (Official Report Entry) with fields for 'Kurum' (Institution) and 'Hekim' (Physician), a text area for 'Klinik değerlendirme' (Clinical evaluation), and a green button 'Raporu Onayla ve Arşivle' (Approve and Archive Report). At the bottom, it displays 'Hasta Protokol: P-164902' and 'Analiz Tarihi: 14 May 2026 - 21:10'.

Findings	Percentage
Consolidation	%17.6
Fibrosis	%16.6
Pleural Thickening	%14.1
Nodule	%13.7
Pneumothorax	%8.4
Pneumonia	%8.3

Resmi Rapor Kaydı

Kurum: Ankara Bilkent Şehir Hastanesi

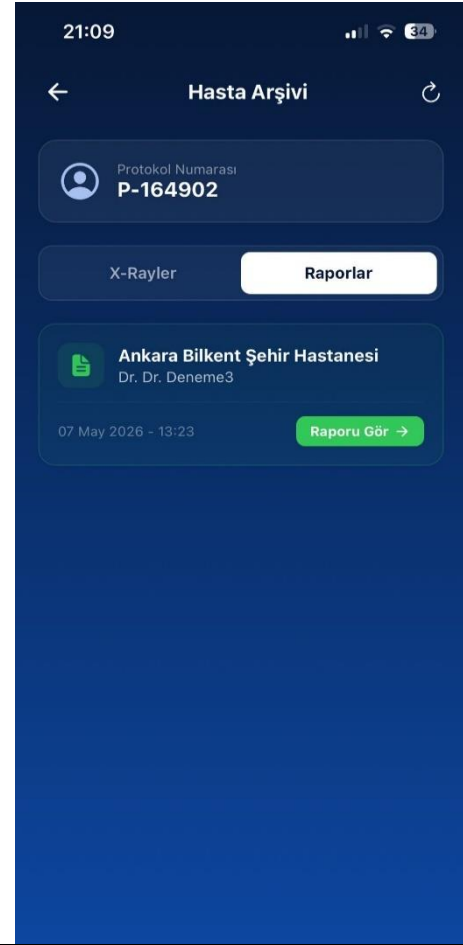
Hekim: Dr. Deneme3

Klinik değerlendirme buraya yazın...

Raporu Onayla ve Arşivle

Hasta Protokol: P-164902
Analiz Tarihi: 14 May 2026 - 21:10

Figure 24: Physician Reporting Interface and Clinical Note Entry



The interface shows a patient's medical history. It includes a header 'Hasta Arşivi' (Patient Archive) with a back arrow and a refresh icon. Below this is a section for 'Protokol Numarası P-164902' with a patient icon. There are two tabs: 'X-Rayler' and 'Raporlar'. Below the tabs is a section for 'Ankara Bilkent Şehir Hastanesi' with 'Dr. Dr. Deneme3' and a date '07 May 2026 - 13:23'. A green button 'Raporu Gör' (View Report) is present.

Hasta Arşivi

Protokol Numarası P-164902

X-Rayler Raporlar

Ankara Bilkent Şehir Hastanesi
Dr. Dr. Deneme3

07 May 2026 - 13:23 Raporu Gör

Figure 25: Patient Archive View for Finalized Medical Reports

The interface is designed to transform complex data into a streamlined reporting workflow, allowing the physician to finalize a diagnosis with minimal effort. As shown in the side-by-side comparison, once the analysis is complete, the doctor can easily review the results, add clinical notes, and generate a formalized report directly within the application. These reports are then instantly synchronized with the patient's dedicated profile. This integration ensures that the physician can access all historical diagnoses and previous X-ray reports through a simplified patient panel, facilitating a quick and organized review of the patient's medical journey without navigating through complex menus.

5.4.8. Integrated Digital Archiving and Diagnostic History

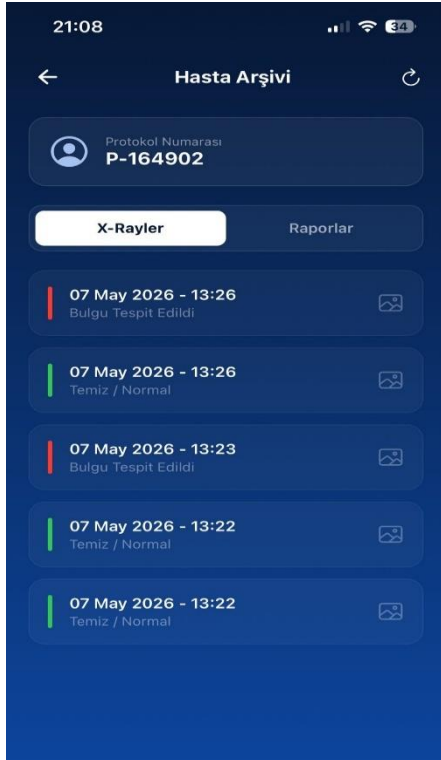


Figure 26: Longitudinal X-Ray Analysis History with Status Indicators



Figure 27: Main Dashboard Featuring Recent Diagnostic Outcomes

The frontend architecture establishes a robust digital archiving system that centralizes all diagnostic activities for long-term patient management. As shown in the side-by-side interface comparison, the system automatically categorizes every finalized analysis into a structured "Patient Archive," where users can toggle between raw X-ray results and official medical reports. This dual-view organization is complemented by a "Recent Analyses" feed on the main dashboard, which provides immediate visibility into the most recent diagnostic outcomes, such as "Findings Detected" (marked in red) or "Clear/Normal" (marked in green). By unifying these two views, the interface allows physicians to monitor a patient's progress over time with high visual clarity, ensuring that critical historical data is always accessible for comparative evaluation.

5.4.9. Lessons Learned

The development of the mobile interface provided several insights into the importance of intuitive design in medical software:

- **The Power of Visual Transparency:** One of the most significant takeaways was that raw data is often overwhelming in clinical settings. By translating complex AI outputs into visual contours and color-coded markers, the interface reduces the doctor's cognitive load. Providing a clear visual "why" behind an AI suggestion is essential for building trust in digital diagnostic tools.
- **Context-Aware Interactivity:** Implementing conditional logic, where diagnostic selections only become active if the AI-detected probability exceeds specific thresholds, proved vital for preventing errors. This ensures that the physician only interacts with clinically significant findings, maintaining focus on the most relevant pathologies.
- **UX for High-Stress Environments:** The choice of a dark-themed UI and high-contrast status indicators (Red for findings, Green for normal results) demonstrated how design can directly support a radiologist's focus. A simplified, streamlined workflow minimizes the time spent on administrative tasks and maximizes time for clinical evaluation.
- **Structured Access and Verification:** Separating doctor and hospital access proved essential for improving role clarity and reducing confusion during login and onboarding. Implementing a hospital based approval mechanism increased system reliability by ensuring that newly registered doctors could not directly access clinical functions without institutional verification. Designing a simple and guided progression from registration to approval and login improved usability and helped users understand their current system status at each stage.

5.4.10. Future Work

The future roadmap for the application's frontend aims to enhance clinical efficiency and diagnostic precision through advanced interactive features. A primary objective is the transition from static image preprocessing to a Dynamic Augmentation Layer, allowing physicians to manually adjust parameters like contrast (CLAHE) and brightness (Gamma Correction) in real-time to better visualize subtle medical signs. Additionally, we plan to implement a Comparative Multi-View Interface supporting the simultaneous display of Posterior-Anterior (PA) and Anterior-Posterior (AP) views. To support the "physician-in-the-loop" model, future iterations will include an AI Feedback Loop, where doctor corrections and manual annotations are captured through the UI for continuous model refinement. Finally, we aim to integrate Predictive Trend Visualization, using historical archive data to generate visual charts of disease progression to help clinicians monitor treatment efficacy.

Additionally, future improvements for this module may include stronger account verification mechanisms such as e-mail confirmation, password recovery, and multi-factor authentication. The hospital approval panel can be extended with advanced filtering, search,

and status history features for better administrative control. Another possible enhancement is the integration of notification support, allowing doctors to be informed automatically when their accounts are approved or rejected. These additions will make the authentication workflow more secure, scalable, and user-friendly.

5.5. Database Design and Implementation

In this project, the database layer was one of the main components that supported the overall workflow of the Chest X-Ray Analysis System. The database was not only used for storing information, but also for connecting doctors, hospitals, patients, uploaded X-ray images, and analysis records in a structured way. MongoDB Atlas was selected as the cloud-based database solution because it provides a flexible document-based structure. This flexibility was useful for our project since medical records, doctor information, patient data, and analysis results may require new fields in future versions of the system.

The selection of MongoDB Atlas as the database system was based on several practical and technical factors. First, the team members responsible for the backend had prior experience with MongoDB, which reduced the learning curve and allowed faster development. Second, MongoDB Atlas provides a free-tier cloud hosting option, which made it accessible for an academic project without additional infrastructure cost. Third, MongoDB's document-based NoSQL structure was suitable for this project because different collections such as users, patients, and analyses have varying field requirements. A relational database with a fixed schema would have required additional migration steps each time a new field was needed, whereas MongoDB allowed these changes without restructuring the entire database. Finally, the availability of extensive documentation and community support made it easier to resolve issues during development.

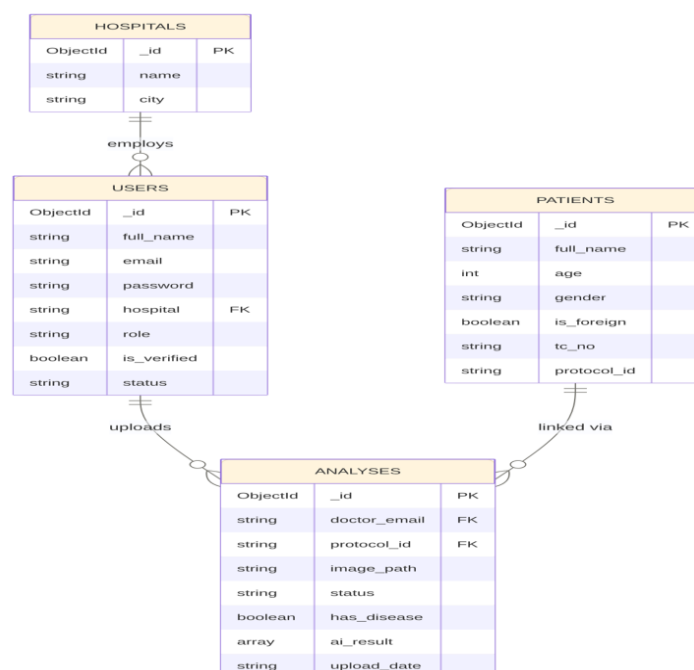


Figure 28: Collection schema of the MongoDB database.

The database was tested with approximately 25 to 50 records distributed across the four collections: users, patients, hospitals, and analyses. The “users” collection stores 8 fields per document including authentication and approval status fields, while the “patients” collection stores 7 fields covering demographic and identification information. The hospitals collection holds 4 fields per document, and the analyses collection stores up to 8 fields per record depending on the processing status. The total data size of the ceng408 database reached 24.75 MB during testing, with 4 indexes maintained across the collections. For completed analyses, the “ai_result” field stores an array of 14 disease classification scores, each representing a confidence value for a specific chest pathology such as Pneumonia, Cardiomegaly, or No Finding. This structure allows the system to preserve the full output of the AI model rather than only the top prediction, which supports more detailed clinical evaluation in future use.

Another contribution is the connection between patient records and analysis records through the protocol ID. When a patient is added to the system, a unique protocol ID is generated automatically. This protocol ID is later used while uploading an X-ray image, so the uploaded image can be linked to the correct patient. This design helps prevent confusion between patients and makes the analysis process more traceable. The patient module also checks whether a T.C. number already exists before creating a new patient, which helps reduce duplicate records.

The database also contributes to system security. During doctor registration, passwords are not stored directly as plain text. Instead, bcrypt is used to hash the password before saving it to the database. In addition, doctor accounts are first created with a pending status and are not allowed to log in until they are approved by an administrator. This approval mechanism is important because the system handles medical-related information, and unauthorized access should be prevented.

Another important contribution is the validation performed before saving an analysis record. Before an X-ray image is uploaded, the backend checks whether the entered protocol ID belongs to an existing patient. If there is no patient with that protocol ID, the system does not accept the image. The backend also checks whether the uploaded file is an image file. After these controls, the image path, doctor email, protocol ID, analysis status, AI result field, and upload date are saved in the analyses collection.

5.5.1. Applications

The database structure directly supports the real usage scenario of the Chest X-Ray Analysis System. A doctor can register to the system, wait for admin approval, log in after approval, add patients, upload chest X-ray images, and track analysis records. Each uploaded X-ray image can be connected to both the doctor who uploaded it and the patient it belongs to. This makes the system useful for medical environments where patient records and image analysis results must be stored in an organized way. Figure 1 illustrates this workflow and the corresponding database operations at each step.

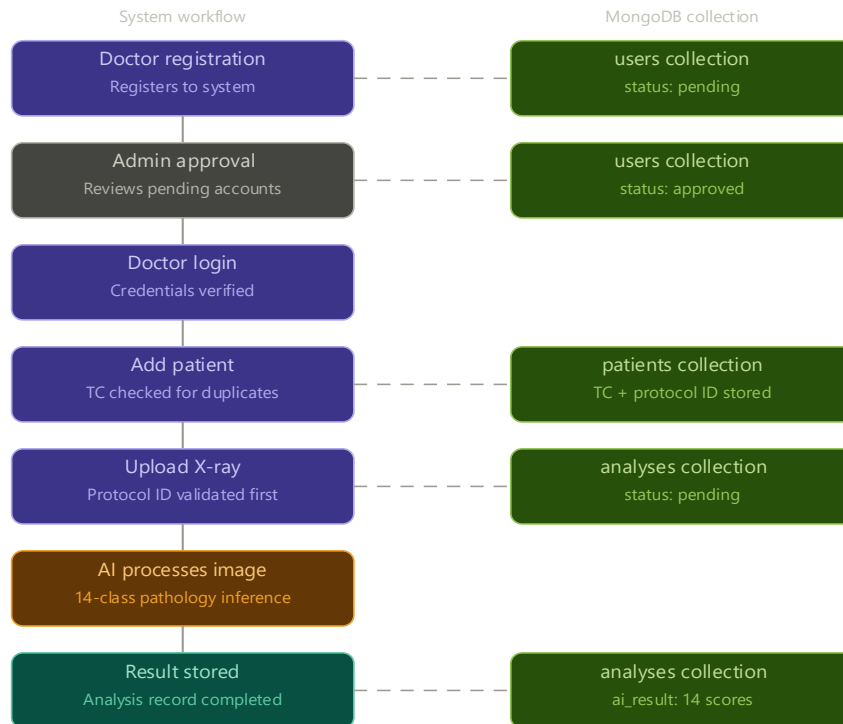


Figure 29: workflow and the corresponding database operations at each step

The hospital information stored in the database also supports the registration process. Since doctors are associated with hospitals, the system can later be extended for hospital-based filtering, hospital-specific doctor lists, or hospital-based analysis statistics. In this way, the current database design can support not only the existing application, but also more advanced healthcare management features in the future.

5.5.2. Lessons Learned

During the development of the database part, we learned that database design is not only about saving data. It also affects security, data consistency, and the usability of the whole application. One important lesson was the need to define clear relationships between different parts of the system. Since MongoDB is not a relational database like SQL databases, these relationships had to be managed carefully through fields such as email and protocol ID.

Another lesson was the importance of validation before inserting data into the database. For example, checking duplicate patients, checking whether a protocol ID exists before uploading an X-ray image, and checking the image file type helped make the system more reliable. Without these controls, the database could contain incorrect or unrelated records.

We also learned that authentication data must be handled carefully. Storing passwords directly would be unsafe, so password hashing became an important part of the database design. Similarly, the admin approval process showed us that user access should be controlled, especially in applications related to healthcare data.

One specific design decision that shaped the database structure was the use of two separate identification fields for patients. Initially, the T.C. identity number was considered sufficient for patient identification. However, since T.C. numbers are sensitive personal data and should not be exposed in general application flows such as image uploads or analysis tracking, a separate protocol ID field was introduced. The T.C. number is only used during patient registration to prevent duplicate records, while the protocol ID is used for all subsequent operations. This separation helped balance data privacy with system usability.

Another decision that was added during development was the doctor approval mechanism. As the system handles medical data, it became clear that allowing any registered user to immediately access patient records would be a security risk. Therefore, a status field was added to the users collection, and doctors are required to wait for administrator approval before they can log in. This change required updating both the registration and login logic, and showed that security requirements in healthcare applications often emerge more clearly once the full workflow is considered.

5.5.3. Future Work

As future work, the database layer can be improved in several ways. First, the MongoDB connection string should be moved from the source code to an environment file. This would make the system more secure because database credentials should not be written directly inside the code.

Second, indexes can be added to frequently searched fields such as email, protocol ID, and T.C. number. This would improve query performance, especially when the number of users, patients, and analysis records increases.

Third, the analysis collection can be expanded. Currently, the system stores the image path, analysis status, AI result field, and upload date. In the future, this structure can be improved by adding doctor comments, final diagnosis confirmation by the physician, and a structured report output. Although the AI model currently returns confidence scores for 14 pathology classes, the system does not yet support doctor annotations or official report generation linked to each analysis. This would allow the system to keep a more complete medical analysis history for each patient.

Finally, the database can be improved to support patient history tracking. For example, a doctor could view all previous X-ray analyses of the same patient through the protocol ID. This would make the system more useful in real clinical scenarios because doctors often need to compare current and previous medical records.

Overall, the database layer provides a secure, organized, and extendable foundation for the Chest X-Ray Analysis System. It supports user authentication, doctor approval, patient registration, hospital data management, X-ray upload control, and analysis tracking. Therefore, the database is not only a storage unit in this project; it is one of the main components that controls the flow and reliability of the whole system.

6. Conclusion

We have successfully developed a model capable of differentiating between illness, we have successfully implemented modified visualization techniques for our model outputs and developed an app for possible users. Through multiple experiments we increased our model's initial average macro AUC score of 0.8111 up to 0.8298 and increased the macro F1 from 0.2736 up to 0.3592. The integration of GradCAM++ contours has successfully provided explainability to our model. We have also managed to decrease the rate of false negatives through the ASL training method and the use of dynamic thresholding. Although we have successfully implemented all aspects of this project, further improvements are still possible through experimentation and more precise thresholding methods.

7. References

- [1] National Institutes of Health, “NIH Chest X-rays Dataset,” Kaggle, [Online].
Available: <https://www.kaggle.com/datasets/nih-chest-xrays/data>.
Accessed: 19 Dec. 2025.
- [2] A. Giełczyk, A. Marciniak, M. Tarczewska & Z. Lutowski, “Pre-processing methods in chest X-ray image classification,” *PLOS ONE*, vol. 17, no. 4, e0265949, Apr. 2022.
Available: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0265949>
Accessed: 19 Dec. 2025.
- [3] IEEE Xplore, “Article (document no. 10955260),” [Online].
Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10955260>.
Accessed: 19 Dec. 2025.
- [4] “Median Filter – an overview,” ScienceDirect Topics, Elsevier, [Online].
Available: <https://www.sciencedirect.com/topics/engineering/median-filter>.
Accessed: 19 Dec. 2025.
- [5] N. F. Sahib and Z. A. Hashim, “Contrast Image Enhancement by Gamma Correction,” *Computer Engineering and Intelligent Systems*, vol. 9, no. 7, 2018.
Available: <https://scispace.com/pdf/contrast-image-enhancement-by-gamma-correction-3upr34p2k1.pdf>.
Accessed: 19 Dec. 2025.
- [6] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely Connected Convolutional Networks,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, [Online].
Available: <https://arxiv.org/abs/1608.06993>
Accessed: 20 Dec. 2025.
- [7] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, [Online].
Available: <https://arxiv.org/abs/1807.06521>
Accessed: 20 Dec. 2025.
- [8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” *IEEE*

International Conference on Computer Vision (ICCV), 2017, [Online].

Available: <https://arxiv.org/abs/1610.02391>

Accessed: 19 Dec. 2025.

[9] X. Wang et al., “ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases,” *CVPR*, 2017.

[10] I. E. Ihongbe et al., “Evaluating Explainable Artificial Intelligence (XAI) techniques in chest radiology imaging through a human-centered lens,” *PLOS ONE*, 2024.

[11] J. Irvin et al., “CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison,” *AAAI*, 2019.

[12] J. Han et al., “BO-CLAHE enhancing neonatal chest X-ray image quality for improved lesion classification,” *Scientific Reports*, 2025.

[13] Flutter Team. Flutter Documentation.

<https://docs.flutter.dev>

[14] Google Developers. Flutter Architectural Overview.

<https://docs.flutter.dev/resources/architectural-overview>

[15] ISO/IEC 25010:2011. Systems and software Quality Requirements and Evaluation (SQuaRE).

[16] ECMA International. (2017). The JSON Data Interchange Syntax (ECMA-404).

[17] Mozilla Developer Network (MDN). HTTP Overview and Status Codes.